

Course Outcomes (CO's)

C01: Analyze and measure performance of sequential and parallel algorithms.

C02: Design and Implement solutions for multicore/Distributed/parallel environment.

C03: Identify and apply the suitable algorithms to solve AI/ML problems. Identify and apply the suitable algorithms to solve AI/ML problems.

C04: Apply the technique of Deep Neural network for implementing linear regression and classification

C05: Apply the technique of Convolution (CNN) for implementing Deep Learning models

C06: Design and develop Recurrent Neural Network (RNN) for prediction.

List of Assignments

Sr.no	Name of Experiment
Part I High Performance Computing	
1	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .
2	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms
3	Implement Min, Max, Sum and Average operations using Parallel Reduction.
4	Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C
5	Implement HPC application for AI/ML domain.
6	Mini Project: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI
7	Mini Project: Implement Huffman Encoding on GPU
8	Mini Project: Implement Parallelization of Database Query optimization
9	Mini Project: Implement Non-Serial Polyadic Dynamic Programming with GPU Parallelization
Part II Deep Learning	
1	Linear regression by using Deep Neural network: Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.
2	Classification using Deep neural network (Any One from the following) 1. Multiclass classification using Deep Neural Networks: Example: Use the OCR letter recognition dataset https://archive.ics.uci.edu/ml/datasets/letter+recognition 2. Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset
3	Convolutional neural network (CNN) (Any One from the following) · Use any dataset of plant disease and design a plant disease detection system using CNN. · Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.
4	Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN
5	Mini Project: Human Face Recognition
6	Mini Project: Gender and Age Detection: predict if a person is a male or female and also their age
7	Mini Project: Colorizing Old B&W Images: color old black and white images to colorful images

Part I High Performance Computing

Group A

Experiment No :- 1

Objective: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS

Outcome: Students will be understand the implementation of Binary search and BFS, DFS

Pre-requisites:

64-bit Open source Linux or its derivative

Programming Languages: C++/JAVA/PYTHON/R

Theory:

Binary Search:

In computer science, binary search, also known as half- interval search, logarithmic search, or binary chop, is a search algorithm that finds the position of a target value within a sorted array. Binary search compares the target value to the middle element of the array. If they are not equal, the half in which the target cannot lie is eliminated and the search continues on the remaining half, again taking the middle element to compare to the target value, and repeating this until the target value is found. If the search ends with the remaining half being empty, the target is not in the array. Even though the idea is simple, implementing binary search correctly requires attention to some subtleties about its exit conditions and midpoint calculation.

Binary search runs in logarithmic time in the worst case, making $O(\log n)$ comparisons where n is the number of elements in the array, the O is Big O notation, and $\log$ is the logarithm. Binary search takes constant ($O(1)$) space, meaning that the space taken by the algorithm is the same for any number of elements in the array. Binary search is faster than linear search except for small arrays, but the array must be sorted first. Although specialized data structures designed for fast searching, such as hash tables, can be searched more efficiently, binary search applies to a wider range of problems.

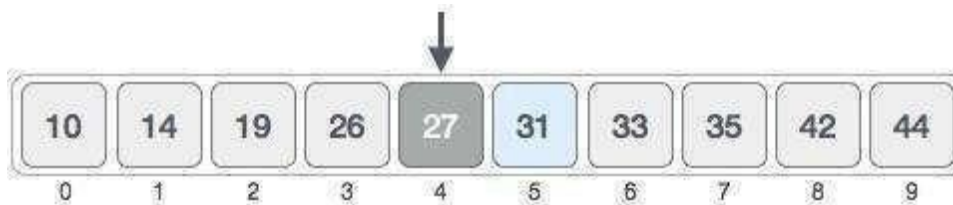
How Binary Search Works?

For a binary search to work, it is mandatory for the target array to be sorted. We shall learn the process of binary search with a pictorial example. The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.

First, we shall determine half of the array by using this formula

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Here it is, $0 + (9 - 0) / 2 = 4$ (integer value of 4.5). So, 4 is the mid of the array.



Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.



We change our low to mid + 1 and find the new mid value again.

$$\text{low} = \text{mid} + 1$$

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.

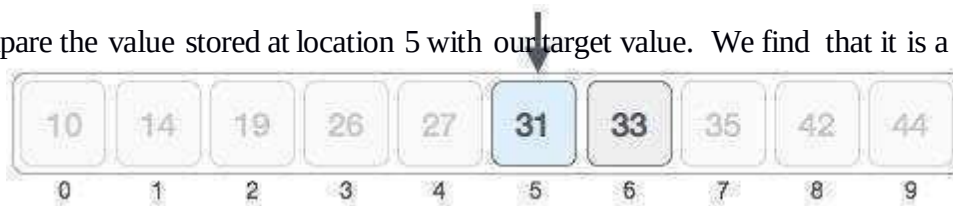


The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.



Hence, we calculate the mid again. This time it is 5.

We compare the value stored at location 5 with our target value. We find that it is a match.



10	14	19	26	27	31	33	35	42	44
0	1	2	3	4	5	6	7	8	9

We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Binary Search Code:

```
#include<iostream>
```

```
#include<stdlib.h>
```

```
#include<omp.h> using
namespace std;
```

```
int binary(int *, int, int, int);
```

```
int binary(int *a, int low, int high, int key){ int mid; mid=(low+high)/2;
```

```
int low1,low2,high1,high2,mid1,mid2,found=0,loc=-1; #pragma omp parallel sections { #pragma omp
section
```

```
{ low1=low; high1=mid; while(low1<=high1) {
```

```
if(!(key>=a[low1] && key<=a[high1])) { low1=low1+high1; continue;
```

```
} cout<<"here1"; mid1=(low1+high1)/2;
```

```
if(key==a[mid1]) { found=1; loc=mid1; low1=high1+1; } else if(key>a[mid1]) { low1=mid1+1; } else
if(key<a[mid1]) high1=mid1-1
}
```

```
#pragma omp section {
```

```
low2=mid+1; high2=high;
while(low2<=high2) {
```

```
if(!(key>=a[low2] && key<=a[high2])) { low2=low2+high2; continue; } cout<<"here2";
```

```

mid2=(low2+high2)/2;

if(key==a[mid2]) { found=1; loc=mid2;

low2=high2+1; } else if(key>a[mid2]) {
low2=mid2+1; } else if(key<a[mid2])
high2=mid2-1;

} } }

return loc;

}

int main(){

int *a,i,n,key,loc=-1;

cout<<"\n enter total no of elements=>"; cin>>n; a=new int[n]; cout<<"\n enter elements=>";
for(i=0;i<n;i++) { cin>>a[i]; } cout<<"\n enter key to find=>"; cin>>key; loc=binary(a,0,n-1,key);

if(loc==-1) cout<<"\n Key not found.";
else cout<<"\n Key found at
position=>"<<loc+1; return 0;

}

```

Breadth-First Search :

Graph traversals

Graph traversal means visiting every vertex and edge exactly once in a well-defined order. While using certain graph algorithms, you must ensure that each vertex of the graph is visited exactly once. The order in which the vertices are visited are important and may depend upon the algorithm or question that you are solving. During a traversal, it is important that you track which vertices have been visited. The most common way of tracking vertices is to mark them.

Breadth First Search (BFS)

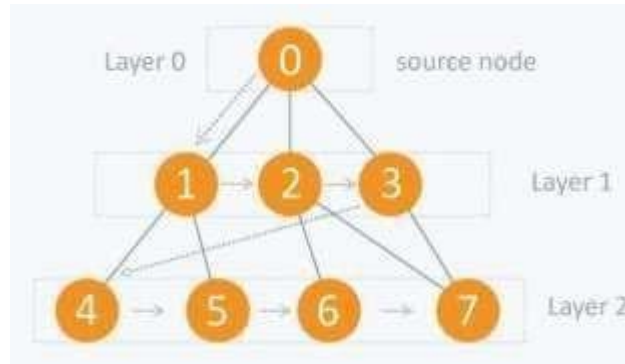
There are many ways to traverse graphs. BFS is the most commonly used approach.

BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layerwise thus exploring the neighbour nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbour nodes.

As the name BFS suggests, you are required to traverse the graph breadthwise as follows:

First move horizontally and visit all the nodes of the current layer

Move to the next layer



Consider the following diagram.

The distance between the nodes in layer 1 is comparatively lesser than the distance between the nodes in layer 2. Therefore, in BFS, you must traverse all the nodes in layer 1 before you move to the nodes in layer

Traversing child nodes

A graph can contain cycles, which may bring you to the same node again while traversing the graph. To avoid processing of same node again, use a boolean array which marks the node after it is processed. While visiting the nodes in the layer of a graph, store them in a manner such that you can traverse the corresponding child nodes in a similar order.

In the earlier diagram, start traversing from 0 and visit its child nodes 1, 2, and 3. Store them in the order in which they are visited. This will allow you to visit the child nodes of 1 first (i.e. 4 and 5), then of 2 (i.e. 6 and 7), and then of 3 (i.e. 7) etc.

To make this process easy, use a queue to store the node and mark it as 'visited' until all its neighbours (vertices that are directly connected to it) are marked. The queue follows the First In First Out (FIFO) queuing method, and therefore, the neighbors of the node will be visited in the order in which they were inserted in the node i.e. the node that was inserted first will be visited first, and so on.

Pseudo code for BFS:

```
FS (G, s) //Where G is the graph and s is the source node let
Q be queue.
Q.enqueue( s ) //Inserting s in queue until all its neighbour vertices are marked.
marks as visited.
while ( Q is not
empty)
//Removing that vertex from queue, whose neighbour will be visited now v =
Q.dequeue( )
//processing all the
neighbours of v for all
neighbours w of v in Graph G
if w is not visited
```


Q.enqueue(w) //Stores w in Q to further visit its neighbour mark w as visited.

Traversing process

Code for BFS:

```
#include<iostream>
```

```
#include<stdlib.h>
```

```
#include<queue> using
```

```
namespace std;
```

```
class node{ public: node *left, *right; int data;};
```

```
class Breadthfs{ public: node *insert(node *, int); void bfs(node *); }; node *insert(node *root, int data){ //inserts a node in tree if(!root) {
```

```
root=new node; root-
```

```
>left=NULL; root-
```

```
>right=NULL; root-
```

```
>data=data; return root;
```

```
}
```

```
queue<node *> q; q.push(root);
```

```
while(!q.empty()) {
```

```
node *temp=q.front(); q.pop();
```

```
SIT Lonavala
```

```
if(temp->left==NULL) {
```

```
temp->left=new node; temp->left-
```

```
>left=NULL; temp->left->right=NULL ;
```

```
temp->left->data=data; return root;
```

```
}
```

```
else { q.push(temp->left); }
```

```
if(temp->right==NULL) { temp-
```

```
>right=new node; temp->right-
```

```
>left=NULL; temp->right-
```

```
>right=NULL; temp->right-
```

```
>data=data;
```

```
return root; }
```

```
else { q.push(temp->right); }
```

```
} }
```

```
void bfs(node *head){
```

```
queue<node*> q; q.push(head); int
```

```

qSize; while (!q.empty()) {
qSize = q.size();
#pragma omp parallel for //creates
parallel threads for (int i = 0; i <
qSize; i++) { node* currNode; #pragma
omp critical {
currNode = q.front();
op(); cout<<"\t"<<currNode-
>data; } //prints parent node
#pragma omp critical {

if(currNode->left) q.push(currNode->left); //push parent's left node in queue if(currNode->right)
q.push(currNode->right); } //push parent's right node in queue } }

}
int main(){

node *root=NULL; int data; char
ans; do { cout<<"\n enter
data=>"; cin>>data;
root=insert(root,data);

cout<<"do you want insert one more node?"; cin>>ans;
}while(ans=='y'||ans=='Y');

bfs(root);
return 0;
}

```

Conclusion :- In this way we can achieve parallelism while implementing BFS

Experiment No:2

Title: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms..

Aim: Understand Parallel Sorting Algorithms like Bubble sort and Merge Sort.

Prerequisites:

Student should know basic concepts of Bubble sort and Merge Sort.

Objective: Study of Parallel Sorting Algorithms like Bubble sort and Merge Sort

Theory:

i) What is Sorting?

Sorting is a process of arranging elements in a group in a particular order, i.e., ascending order, descending order, alphabetic order, etc.

- Arrange elements of a list into certain order
- Make data become easier to access
- Speed up other operations such as searching and merging. Many sorting algorithms with different time and space complexities.

ii) What is Parallel Sorting?

A sequential sorting algorithm may not be efficient enough when we have to sort a huge volume of data. Therefore, parallel algorithms are used in sorting.

- Based on an existing sequential sort algorithm
 - Try to utilize all resources available
 - Possible to turn a poor sequential algorithm into a reasonable parallel algorithm (bubble sort and parallel bubble sort)
 - Completely new approach
 - New algorithm from scratch
 - Harder to develop

- Sometimes yield better solution

Bubble Sort

The idea of bubble sort is to compare two adjacent elements. If they are not in the right order, switch them. Do this by comparing and switching (if necessary) until the end of the array is reached. Repeat this process from the beginning of the array n times.

- One of the straight-forward sorting methods
- Cycles through the list
- Compares consecutive elements and swaps them if necessary
- Stops when no more out of order pair
 - Slow & inefficient
 - Average performance is $O(n^2)$

Bubble Sort Example

Here we want to sort an array containing [8, 5, 1]. The following figure shows how we can sort this array using bubble sort. The elements in consideration are shown in **bold**.

- Implemented as a pipeline.
- Let $local_size = n / no_proc$. We divide the array in no_proc parts, and each process executes the bubble sort on its part, including comparing the last element with the first one belonging to the next thread.
- Implement with the loop (instead of $j < i$)

for ($j=0$; $j < n-1$; $j++$)

- For every iteration of i , each thread needs to wait until the previous thread has finished that iteration before starting.
- We'll coordinate using a barrier.

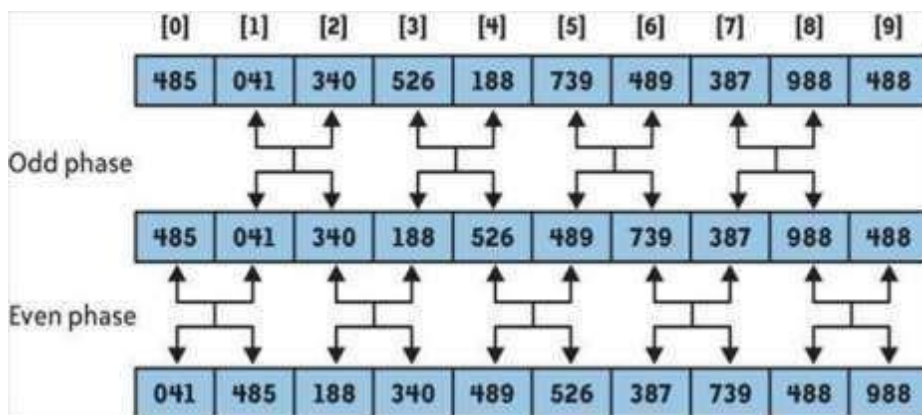
Algorithm for Parallel Bubble Sort

1. For $k = 0$ to $n-2$
2. If k is even then
3. for $i = 0$ to $(n/2)-1$ do in parallel
4. If $A[2i] > A[2i+1]$ then

5. Exchange $A[2i] \leftrightarrow A[2i+1]$
6. Else
7. for $i = 0$ to $(n/2)-2$ do in parallel
8. If $A[2i+1] > A[2i+2]$ then
9. Exchange $A[2i+1] \leftrightarrow A[2i+2]$
10. Next k

Parallel Bubble Sort Example 1

- Compare all pairs in the list in parallel
- Alternate between odd and even phases
- Shared flag, **sorted**, initialized to true at beginning of each iteration (2 phases), if any processor perform swap, **sorted** = false



Parallel Bubble Sort Example 2

- How many steps does it take to sort the following sequence from least to greatest using the Parallel Bubble Sort? How does the sequence look like after 2 cycles?
- Ex: 4,3,1,2

Merge Sort

- Collects sorted list onto one processor

- Merges elements as they come together
- Simple tree structure
- Parallelism is limited when near the root

Theory:

To sort $A[p \dots r]$:

1. Divide Step

If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split $A[p \dots r]$ into two subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$, each containing about half of the elements of $A[p \dots r]$. That is, q is the halfway point of $A[p \dots r]$.

2. Conquer Step

Conquer by recursively sorting the two subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$.

3. Combine Step

Combine the elements back in $A[p \dots r]$ by merging the two sorted subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$ into a sorted sequence. To accomplish this step, we will define a procedure MERGE (A, p, q, r).

Parallel Merge Sort

- Parallelize processing of sub-problems
- Max parallelization achieved with one processor per node (at each layer/height)

Parallel Merge Sort Example

- Perform Merge Sort on the following list of elements. Given 2 processors, P_0 & P_1 , which processor is responsible for which comparison?
- 4,3,2,1

Algorithm for Parallel Merge Sort

1. Procedure parallelMergeSort
2. Begin
3. Create processors P_i where $i = 1$ to n

4. if $i > 0$ then receive size and parent from the root
5. receive the list, size and parent from the root
6. endif
7. $midvalue = listsize/2$
8. if both children are present in the tree then
9. send midvalue, first child
10. send $listsize - mid$, second child
11. send list, midvalue, first child
12. send list from midvalue, $listsize - midvalue$, second child
13. call `mergelist(list,0,midvalue,list, midvalue+1,listsize,temp,0,listsize)`
14. store temp in another array list2
15. else
16. call `parallelMergeSort(list,0,listsize)`
17. endif
18. if $i > 0$ then
19. send list, listsize, parent
20. endif
21. end

INPUT:

Array of integer numbers.

OUTPUT:

Sorted array of numbers

ALGORITHM ANALYSIS

1. Time Complexity Of parallel Merge Sort and parallel Bubble sort in best case is (when all data is already in sorted form): $O(n)$
2. Time Complexity Of parallel Merge Sort and parallel Bubble sort in worst case is: $O(n \log n)$
3. Time Complexity Of parallel Merge Sort and parallel Bubble sort in average case

is: $O(n \log n)$

APPLICATIONS

1. Representing Linear data structure & Sequential data organization : structure & files
2. For Sorting sequential data structure

Conclusion:

Thus, we have studied Parallel Bubble and Parallel Merge sort implementation.

Experiment No :- 3

Aim: Implement parallel reduction using Min, Max, Sum and Average Operations.

Objective: To study and implementation of directive based parallel programming model.

Outcome: Students will be understand the implementation of sequential program augmented with compiler directives to specify parallelism.

Pre-requisites: 64-bit Open source Linux or its derivative Programming Languages: C/C++

Theory:

OpenMP:

OpenMP is a set of C/C++ pragmas (or FORTRAN equivalents) which provide the programmer a high-level front-end interface which get translated as calls to threads (or other similar entities). The key phrase here is "higher-level"; the goal is to better enable the

programmer to "think parallel," alleviating him/her of the burden and distraction of dealing

with setting up and coordinating threads. For example, the OpenMP directive. OpenMP Core Syntax:

Most of the constructs in OpenMP are compiler directives:

```
#pragma omp construct [clause [clause]...]
```

Example

```
#pragma omp parallel num_threads(4)
```

Function prototypes and types in the file:

```
#include <omp.h>
```

Most OpenMP constructs apply to a "structured block"

Structured block:

a block of one or more statements surrounded by "{ }", with one point of entry at the top and one point of exit at the bottom.

Following is the sample code which illustrates max operator usage in OpenMP :

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main()
```

```
{
```

```

double arr[10];
omp_set_num_threads(4);
double max_val=0.0;
int i;
for( i=0; i<10; i++)
arr[i] = 2.0 + i;
#pragma omp parallel for reduction(max : max_val)
for( i=0;i<10; i++)
{
printf("&quot;thread id = %d and i = %d&quot;,",
omp_get_thread_num(), i); if(arr[i] > max_val)
{
max_val = arr[i];
}
}
printf("&quot;\nmax_val = %f&quot;,", max_val);
}

```

Following is the sample code which illustrates min operator usage in OpenMP :

```

#include <stdio.h>;
#include <omp.h>;
int main()
{
double arr[10];
omp_set_num_threads(4);
double min_val=0.0;
int i;
for( i=0; i<10; i++)
arr[i] = 2.0 + i;
#pragma omp parallel for reduction(min : min_val)
for( i=0;i<10; i++)
{
printf("&quot;thread id = %d and i = %d&quot;,",
omp_get_thread_num(), i); if(arr[i] < min_val)
{
min_val = arr[i];
}
}
}

```

```

}
printf(&quot;\nmin_val = %f&quot;, min_val);
}

```

Following is the sample code which illustrates sum operation usage in OpenMP :

```

#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[])
{
    int i, n;
    float a[100], b[100], sum;
    /* Some initializations */
    n = 100;
    for (i=0; i < n; i++)
        a[i] = b[i] = i * 1.0;
    sum = 0.0;
    #pragma omp parallel for reduction(+:sum)
    for (i=0; i < n; i++)
        sum = sum + (a[i] * b[i]);
    printf(&quot; Sum = %f\n&quot;,sum);
}

```

Following is the sample code which illustrates sum operation usage in OpenMP

Conclusion: We have implemented parallel reduction using Min, Max, Sum and Average Operations.

Experiment No :- 4

Title:- Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C

Aim: Implement $n \times n$ matrix parallel addition, multiplication using CUDA, use shared memory.

Prerequisites: Concept of matrix addition, multiplication. Basics of CUDA programming

Objectives: Student should be able to learn parallel programming, CUDA architecture and CUDA processing flow

Theory:

A straightforward matrix multiplication example that illustrates the basic features of memory and thread management in CUDA programs

- Leave shared memory usage until later
- Local, register usage
- Thread ID usage
- $P = M * N$ of size $WIDTH \times WIDTH$
- One thread handles one element of P
- M and N are loaded $WIDTH$ times from global memory

Matrix Multiplication steps

1. Matrix Data Transfers
2. Simple Host Code in C
3. Host-side Main Program Code
4. Device-side Kernel Function
5. **Some Loose Ends**

Step 1: Matrix Data Transfers

/ Allocate the device memory where we will copy M to Matrix Md;

```
Md.width = WIDTH;
```

```
Md.height = WIDTH;
```

```
Md.pitch = WIDTH;
```

```
int size = WIDTH * WIDTH * sizeof(float);
```

```
cudaMalloc((void**)&Md.elements, size);
```

```
// Copy M from the host to the device
```

```
cudaMemcpy(Md.elements, M.elements, size, cudaMemcpyHostToDevice);
```

```
Read M from the device to the host into P cudaMemcpy(P.elements, Md.elements, size,  
cudaMemcpyDeviceToHost);
```

```
Free device memory cudaFree(Md.elements);
```

Step 2: Simple Host Code in C

Matrix multiplication on the (CPU) host in double precision

for simplicity, we will assume that all dimensions are equal

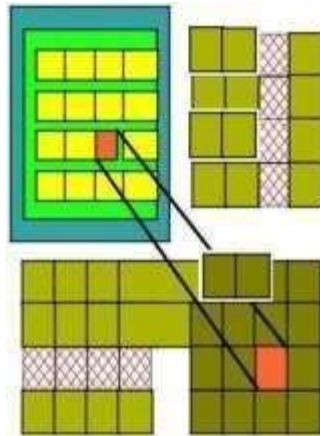
```
void MatrixMulOnHost(const Matrix M, const Matrix N, Matrix P)
```

```
{  
for (int i = 0; i < M.height; ++i)  
  
for (int j = 0; j < N.width; ++j) {  
  
double sum = 0;  
  
for (int k = 0; k < M.width; ++k) {  
  
double a = M.elements[i * M.width + k];  
  
double b = N.elements[k * N.width + j];  
  
sum += a * b;  
  
}  
  
P.elements[i * N.width + j] = sum;  
}
```

Multiply Using One Thread Block

- One Block of threads compute matrix P
 - Each thread computes one element of P
- Each thread
 - Loads a row of matrix M
 - Loads a column of matrix N
 - Perform one multiply and addition for each pair of M and N elements

- Compute to off-chip memory access ratio close to 1:1 (not very high)



- Size of matrix limited by the number of threads allowed in a thread block

Step 3: Host-side Main Program Code

```
int main(void) {
// Allocate and initialize the matrices
Matrix M = AllocateMatrix(WIDTH, WIDTH, 1);
Matrix N = AllocateMatrix(WIDTH, WIDTH, 1);
Matrix P = AllocateMatrix(WIDTH, WIDTH, 0);

    / M * N on the device
    MatrixMulOnDevice(M, N, P);

    / Free matrices
    FreeMatrix(M)

;
FreeMatrix(N)
;
FreeMatrix(P);
return 0;
}
```

Host-side code

```
// Matrix multiplication on the device
```

```

void MatrixMulOnDevice(const Matrix M, const Matrix N, Matrix P)
{
    Load M and N to the device Matrix Md = AllocateDeviceMatrix(M); CopyToDeviceMatrix(Md, M);
    Matrix Nd = AllocateDeviceMatrix(N); CopyToDeviceMatrix(Nd, N);

    Allocate P on the device

    Setup the execution configuration dim3 dimBlock(WIDTH, WIDTH);

    dim3 dimGrid(1, 1);

    Launch the device computation threads! MatrixMulKernel<<<dimGrid, dimBlock>>>(Md, Nd, Pd);

    Read P from the device

    CopyFromDeviceMatrix(P, Pd)
    Free device matrices
    FreeDeviceMatrix(Md);

    FreeDeviceMatrix(Nd);

    FreeDeviceMatrix(Pd);

}

```

Step 4: Device-side Kernel Function

```

// Matrix multiplication kernel – thread specification global void MatrixMulKernel(Matrix M, Matrix N,
Matrix P)
{

// 2D Thread ID
int tx = threadIdx.x;
int ty = threadIdx.y;
Pvalue is used to store the element of the matrix
that is computed by the thread
float Pvalue = 0;
for (int k = 0; k < M.width; ++k)
{
float Melement = M.elements[ty * M.pitch + k]; float Nelement = Nd.elements[k * N.pitch + tx]; Pvalue
+= Melement * Nelement;
}
Write the matrix to device memory;

    / each thread writes one element P.elements[ty

```

```
/    *P.pitch + tx] = Pvalue;
```

```
}
```

Facilities:

Latest version of 64 Bit Operating Systems, CUDA enabled NVIDIA Graphics card

Input:

Two matrices

Output:

Multiplication of two matrix

Conclusion:- We learned parallel programming with the help of CUDA architecture

Experiment No :-5

Problem Statement: Implement HPC application for AI/ML domain

Objectives: To understand HPC application.

Hardware and Software Requirements:

- Ubuntu 16.04 and above
- TensorFlow or PyTorch, or Caffe

Theory:

One possible high-performance computing (HPC) application for the artificial intelligence/machine learning (AI/ML) domain is training deep neural networks on large datasets. Deep learning models have millions of parameters that need to be optimized during training, which can take a long time on a single CPU. HPC systems can accelerate this process by distributing the training workload across multiple processors or even multiple nodes in a cluster. Here is a brief overview of the steps involved in implementing an HPC application for training deep neural networks:

Choose a deep learning framework: There are many popular deep learning frameworks, such as TensorFlow, PyTorch, and Caffe. Choose one that has good support for distributed training and is compatible with your HPC system.

Prepare the data: The data used for training the neural network should be stored in a format that can be efficiently accessed by the HPC system, such as HDF5 or LMDB. The data should also be split into batches that can be processed in parallel.

Design the neural network architecture: Choose an appropriate neural network architecture that is well-suited to the task at hand. This may involve experimenting with different architectures and hyper parameters to find the best model.

Implement the training algorithm: This involves defining the loss function and optimization method used to train the neural network. Stochastic gradient descent (SGD) with momentum is a popular optimization method for deep learning.

Distribute the training workload: Use a distributed training framework to divide the training workload across multiple processors or nodes in the HPC system. This can be done using data parallelism, model parallelism, or a combination of both. In data parallelism, each processor or node processes a subset of the data and updates the model parameters in parallel. In model parallelism, different processors or nodes are responsible for computing different parts of the model. **Monitor and tune the training process:** Monitor the training process to ensure that it is progressing as expected and that there are no performance bottlenecks. Tune the hyper parameters and architecture as needed to improve the accuracy and convergence speed of the model.

Evaluate the trained model: Once the model is trained, evaluate its performance on a validation dataset to assess its accuracy and generalization ability. The trained model can then be used for prediction on

new data.

Overall, an HPC application for training deep neural networks involves a combination of software engineering, algorithm design, and system administration skills. It requires knowledge of deep learning frameworks, distributed computing, and performance optimization techniques, as well as an understanding of the specific domain and dataset being used. With the right expertise and resources, an HPC application for AI/ML can enable faster, more accurate training of deep learning models on large datasets, leading to new insights and discoveries in the field.

Conclusion:

In this way we have Implemented HPC application for AI/ML domain

Miniprojects

Title: Measuring the performance enhancement of a parallel Quicksort algorithm using MPI

Objective: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI

Hardware and Software Requirements:

- Ubuntu 16.04 and above
- MPI
- C++

Theory:

1. Compare the execution time of the parallel algorithm with that of the sequential algorithm.

1. Divide the input array into equal-sized partitions, and have each process sort its partition using the standard

Quicksort algorithm.

1. The processes then exchange their partitions with each other, and use a parallel merge operation to merge the sorted partitions into a single, sorted array.

2. To evaluate the performance enhancement, run the parallel Quicksort algorithm with different number of processes and compare the execution time with the execution time of the sequential Quicksort algorithm.

1. You can also vary the size of the input array and observe how the execution time changes.

1. Additionally, you can measure the speedup and efficiency of the parallel algorithm by using the following

formulas:

$$\text{Speedup} = T_{\text{seq}} / T_{\text{par}}$$

$$\text{Efficiency} = \text{Speedup} / P$$

Where: T_{seq} = execution time of the sequential algorithm

T_{par} = execution time of the parallel algorithm

P = number of processes used

1. Let's start with recursive decomposition - the list is partitioned serially and each of the subproblems is handled by a different processor.

2. The time for this algorithm is lower-bounded by $\Omega(n)$!

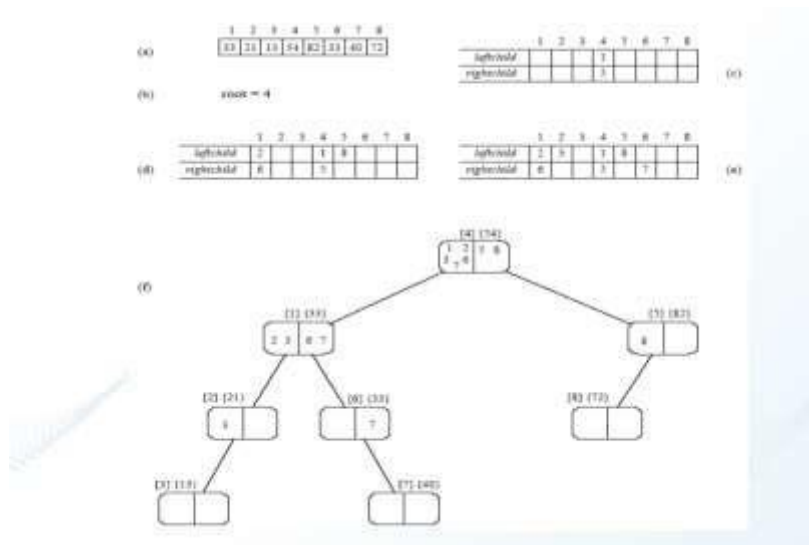
3. Can we parallelize the partitioning step - in particular, if we can use n processors to partition a list of length n around a pivot in $O(1)$ time, we have a winner.

4. This is difficult to do on real machines, though. Parallel Quicksort

PRAM Formulation

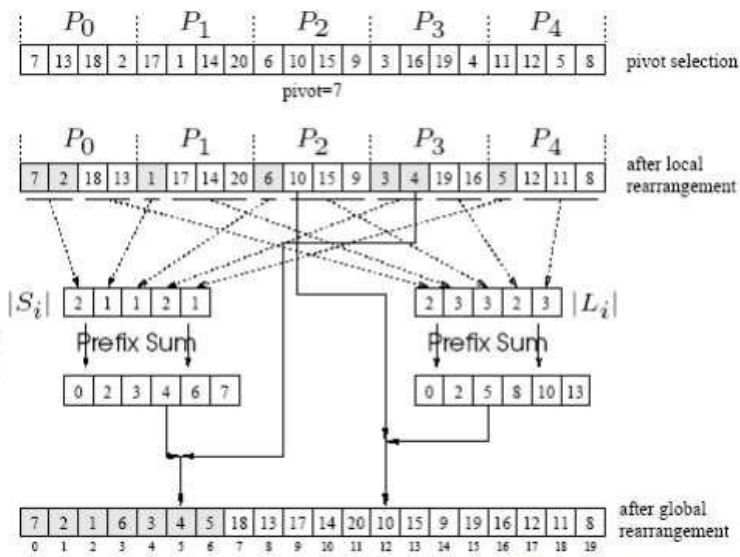
1. We assume a CRCW (concurrent read, concurrent write) PRAM with concurrent writes resulting in an arbitrary write succeeding.
2. The formulation works by creating pools of processors. Every processor is assigned to the same pool initially and has one element.
3. Each processor attempts to write its element to a common location (for the pool).
4. Each processor tries to read back the location. If the value read back is greater than the processor's value, it assigns itself to the 'left' pool, else, it assigns itself to the 'right' pool.
5. Each pool performs this operation recursively.
6. Note that the algorithm generates a tree of pivots. The depth of the tree is the expected parallel runtime. The average value is $O(\log n)$.

PRAM Formulation



Parallelizing Quicksort: Shared Address Space Formulation

1. The only thing we have not described is the global reorganization (merging) of local lists to form L and U.
2. The problem is one of determining the right location for each element in the merged list.
3. Each processor computes the number of elements locally less than and greater than pivot.
4. It computes two sum-scans to determine the starting location for its elements in the merged L and U lists.
5. Once it knows the starting locations, it can write its elements safely.



Parallelizing Quicksort: Shared Address Space Formulation

1. The parallel time depends on the split and merge time, and the quality of the pivot.
2. The latter is an issue independent of parallelism, so we focus on the first aspect, assuming ideal pivot selection.
3. The algorithm executes in four steps: (i) determine and broadcast the pivot; (ii) locally rearrange the array assigned to each process; (iii) determine the locations in the globally rearranged array that the local elements will go to; and (iv) perform the global rearrangement.
4. The first step takes time $\Theta(\log p)$, the second, $\Theta(n/p)$, the third, $\Theta(\log p)$, and the fourth, $\Theta(n/p)$.
5. The overall complexity of splitting an n -element array is $\Theta(n/p) + \Theta(\log p)$.

```
#include <iostream>
#include <algorithm>
#include <mpi.h>
using namespace std;
// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low- 1);
    for (int j = low; j <= high- 1; j++) {

        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
}
```

```

swap(arr[i + 1], arr[high]);
return (i + 1);
}
// Function to perform quicksort on the partition
void quicksort(int arr[], int low, int high) {
if (low < high) {
int pivot = partition(arr, low, high);
quicksort(arr, low, pivot- 1);
quicksort(arr, pivot + 1, high);
}
}
int main(int argc, char *argv[]) {
int rank, size;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
int n = 100;
int* arr = new int[n];
int* recvbuf = new int[n];
int* sendbuf = new int[n];
// Fill the array with random values
if (rank == 0) {
for (int i = 0; i < n; i++) {
arr[i] = rand() % 100;
}
} // Divide the array into equal -sized partitions for each process

int sub_arr_size = n / size;
int* sub_arr = new int[sub_arr_size];
MPI_Scatter(arr, sub_arr_size, MPI_INT, sub_arr, sub_arr_size, MPI_INT, 0, MPI_COMM_WORLD);
// Sort the partition using quicksort
quicksort(sub_arr, 0, sub_arr_size,- 1);
// Gather the sorted partitions from each process
MPI_Gather(sub_arr, sub_arr_size, MPI_INT, recvbuf, sub_arr_size, MPI_INT, 0,
MPI_COMM_WORLD);
// Print the sorted array
for (int i = 0; i < n; i++) {
cout << recvbuf[i] << " ";

```

```

}
cout << endl;
// Measure the execution time of the parallel quicksort algorithm
double start_time = MPI_Wtime();
// Perform the above steps again
double end_time = MPI_Wtime();
double parallel_execution_time = end_time - start_time;
// Measure the execution time of the sequential quicksort algorithm
start_time = MPI_Wtime();
quicksort(arr, 0, n- 1);
end_time = MPI_Wtime();
double sequential_execution_time = end_time - start_time;
// Calculate speedup and efficiency
double speedup = sequential_execution_time /
parallel_execution_time;
double efficiency = speedup / size;
cout << "Sequential execution time: " <<
sequential_execution_time << endl;
cout << "Parallel execution time: " <<
parallel_execution_time << endl;
cout << "Speedup: " << speedup << endl;
cout << "Efficiency: " << efficiency << endl;
}
MPI_Finalize();
return 0;

```

Conclusion: Parallel programming with MPI can significantly improve the performance of certain types of algorithms by distributing the workload across multiple processes

Title: Implement Huffman Encoding on GPU

Objective: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI

Hardware and Software Requirements:

- Ubuntu 16.04 and above
- CUDA Toolkit
- Visual Studio:

Theory:

Huffman encoding is a data compression technique that is used to compress data files.

- Huffman encoding uses a variable-length code for each character in the file.
- The code lengths are determined by the frequency of each character in the file.
- The most common characters are assigned the shortest codes, and the less common characters are assigned longer codes.
- This technique can be used to compress any type of file, but it is most commonly used to compress text files

Advantages of implementing Huffman encoding on a GPU include

- ✓ Parallel processing
- ✓ Improved performance

Disadvantages of implementing Huffman encoding on GPU include

- ✓ Overhead
- ✓ Limited memory

Algorithm :

Step 1: Input data transfer: The input data to be compressed is transferred from the CPU memory to the GPU memory using data transfer techniques like DMA or CUDA's `cudaMemcpy()` function.

Step 2: Frequency counting: The frequency of occurrence of each symbol in the input data is counted on the GPU. This step involves traversing the input data and incrementing the frequency of each symbol encountered. This process can be parallelized by dividing the input data into chunks and assigning each chunk to a separate thread block on the GPU.

Step 3: Building the Huffman tree: Once the frequency counts are obtained, a Huffman tree is constructed on the GPU using a parallel algorithm. This step involves assigning each symbol to a leaf node of the tree and recursively combining the two lowest frequency nodes to form a new internal node until a single root node is formed.

Step 4: Generating the Huffman codes: The Huffman codes are generated on the GPU by traversing the Huffman tree and assigning a bit code to each symbol based on its position in the tree. This step involves maintaining a stack of nodes as the tree is traversed and assigning a 0 or 1 bit to each symbol encountered.

Step 5: Encoding the data: The input data is encoded into its Huffman representation on the GPU using the Huffman codes generated in the previous step. This step involves traversing the input data and replacing each symbol with its corresponding Huffman code. The encoded data is stored in GPU memory.

Step 6: Output data transfer: The encoded data is transferred from the GPU memory back to the CPU memory using data transfer techniques like DMA or CUDA's `cudaMemcpy()` function.

Huffman Encoding Code:


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <limits.h>

#include "../include/parallel.h"
#include "../include/color.h"

#define BLOCK_SIZE 1024
#define MIN_SCRATCH_SIZE 50 * 1024 * 1024
#define DEBUG 1

struct huffmanNode *huffmanTreeNode_head;
struct huffmanDictionary huffmanDictionary;
struct huffmanNode huffmanTreeNode[512];
unsigned char bitSequenceConstMemory[256][255];
unsigned int constMemoryFlag = 0;

int main(int argc, char **argv)
{
    unsigned int distinctCharacterCount, mergedHuffmanNodes, inputFileLength;
    unsigned int frequency[256];
    unsigned char *inputFileData, bitSequenceLength = 0, bitSequence[255];
    unsigned int *compressedDataOffset;
    unsigned int integerOverflowFlag;
    long unsigned int memFree, memTotal, memRequired, memOffset, memData;

    int numberOfKernels;
    clock_t start, end;

    FILE *inputFile, *compressedFile, *timeTakenGPUFile;

    // check the arguments
    if (argc != 3)
    {
        printf("Arguments should be input file and output file");
        return -1;
    }

    // read input file, get length and data
    inputFile = fopen(argv[1], "rb");
    fseek(inputFile, 0, SEEK_END);
    inputFileLength = ftell(inputFile);
    fseek(inputFile, 0, SEEK_SET);
    inputFileData = (unsigned char *)malloc(inputFileLength * sizeof(unsigned char));
    fread(inputFileData, sizeof(unsigned char), inputFileLength, inputFile);
    fclose(inputFile);

```

```

// starting the clock
start = clock();

// find frequency of each symbols
for (int i = 0; i < 256; i++)
frequency[i] = 0;
for (int i = 0; i < inputFileLength; i++)
frequency[inputFileData[i]]++;

// initialize the nodes
distinctCharacterCount = 0;
for (int i = 0; i < 256; i++)
{
if (frequency[i] > 0)
{
huffmanTreeNode[distinctCharacterCount].frequency = frequency[i];
huffmanTreeNode[distinctCharacterCount].letter = i;
huffmanTreeNode[distinctCharacterCount].left = NULL;
huffmanTreeNode[distinctCharacterCount].right = NULL;
distinctCharacterCount++;
}
}

// build the huffman tree
for (int i = 0; i < distinctCharacterCount - 1; i++)
{
mergedHuffmanNodes = 2 * i;
sortHuffmanTree(i, distinctCharacterCount, mergedHuffmanNodes);
buildHuffmanTree(i, distinctCharacterCount, mergedHuffmanNodes);
}
if (distinctCharacterCount == 1)
{
huffmanTreeNode_head = &huffmanTreeNode[0];
}

// build the huffman dictionary
buildHuffmanDictionary(huffmanTreeNode_head, bitSequence, bitSequenceLength);

// calculating memory requirements

// gpu memory
// cudaMemGetInfo(& memFree, & memTotal);
// cudaMemGetInfo(memFree, memTotal);

// debug
// if (DEBUG)
// printf("Free Memory :: %lu \n", memFree);

// offset array requirements

```

```

memOffset = 0;
for (int i = 0; i < 256; i++)
memOffset += frequency[i] * huffmanDictionary.bitSequenceLength[i];
memOffset = memOffset % 8 == 0 ? memOffset : memOffset + 8 - memOffset % 8;

// other memory requirements
memData = inputFileLength + (inputFileLength + 1) * sizeof(unsigned int) +
sizeof(huffmanDictionary);

if (memFree - memData < MIN_SCRATCH_SIZE)
{
printf("\n%sNot enough memory on GPU :: \nMem Free :: %lu\nMin Required ::
%lu\n", COLOR_ERROR, memFree, memData + MIN_SCRATCH_SIZE);
return -1;
}

memRequired = memFree - memData - 10 * 1024 * 1024;
numberOfKernels = ceil((double)memOffset / memRequired);
integerOverflowFlag = memRequired + 255 <= UINT_MAX || memOffset + 255 <=
UINT_MAX ? 0 : 1;

if (DEBUG)

{
printf("\n\t%sInput File Size :: %u\n\tOutput Size :: %lu\n\tNumber of Kernels ::
%d\n\tInteger Overflow flag :: %d\n", COLOR_DEBUG, inputFileLength, memOffset / 8,
numberOfKernels, integerOverflowFlag);
}

// generate offset data array
compressedDataOffset = (unsigned int *)malloc((inputFileLength + 1) * sizeof(unsigned
int));

// launch kernel
launchCudaHuffmanCompress(inputFileData, compressedDataOffset, inputFileLength,
numberOfKernels, integerOverflowFlag, memRequired);

// end the clock, tick tick
end = clock();

// writing the compressed file to the output
compressedFile = fopen(argv[2], "wb");
fwrite(&inputFileLength, sizeof(unsigned int), 1, compressedFile);
fwrite(frequency, sizeof(unsigned int), 256, compressedFile);
fwrite(inputFileData, sizeof(unsigned char), memOffset / 8, compressedFile);
fclose(compressedFile);

float cpuTimeUsed = ((float)(end - start)) / CLOCKS_PER_SEC;
printf("\n\nTime taken by GPU :: %f s\n", cpuTimeUsed);

```

```

float timeTakenVar;
timeTakenVar = cpuTimeUsed;

timeTakenGPUFile = fopen("timeTakenGPU.txt", "w");

// check if the file was opened successfully
if (timeTakenGPUFile == NULL)
{
printf("Error opening the TimeTaken file.\n");
return -1;
}

// write the double variable to the file
fprintf(timeTakenGPUFile, "%f", timeTakenVar);

// close the file
fclose(timeTakenGPUFile);

free(inputFileData);
free(compressedDataOffset);

return 0;
}

```

Output:

```

Letter : v
Encoding : 0100000000
Letter : r
Encoding : 0100000001
Letter : y
Encoding : 0100000100
Letter : q
Encoding : 0101000001
Letter : k
Encoding : 0101000100
Letter : g
Encoding : 0101000101
Letter : a
Encoding : 0101010000
Letter : f
Encoding : 0101010001
Letter : m
Encoding : 0101010100
Letter : c
Encoding : 0101010101

Input File Size :: 52466954
Output Size :: 31819724
Number of Kernels :: 1
Integer Overflow flag :: 0

Free Mem: 0

Time taken by GPU :: 1.371000 s

```

```

PS D:\Forth Year\Sem 8\Lab\LP5\Mini Projects\HPC\Huffman-Compression> ./bin/compress input.txt input.compressed

Compressed file length :: 31805042
Input file length :: 52442313
Merged Huffman Nodes :: 52
Distinct character count :: 28
Time taken by CPU :: 2.926000 s
PS D:\Forth Year\Sem 8\Lab\LP5\Mini Projects\HPC\Huffman-Compression> ./bin/compress_cuda input.txt input.compressed

Free Memory :: 1090915050

Input File Size :: 52442313
Output Size :: 31805042
Number of Kernels :: 1
Integer Overflow flag :: 0

Free Mem: 4051094857

Time taken by GPU :: 1.363000 s
PS D:\Forth Year\Sem 8\Lab\LP5\Mini Projects\HPC\Huffman-Compression> ./speed_up
The Speed up is: 2.14674

```

Conclusion: In this way we can performance enhancement of parallel Quicksort Algorithm using MPI

Title: Parallelization of Database Query optimization

Objective: Implement Parallelization of Database Query optimization

Speed up query execution

- Reduce processing time
- Optimize resource utilization
- Reduce hardware costs

Hardware and Software Requirements:

- Linux/Windows 10 OS
- Python
- Sqlite DB
- Any Text Editor

Theory:

Database query optimization using parallelism is the process of dividing a database query into sub-tasks that can be executed simultaneously on multiple processing units. By using parallelism, we can achieve faster query execution times, reduce processing time, improve database performance, handle large datasets, increase scalability, optimize resource utilization, and reduce hardware costs. The key steps in optimizing database queries using parallelism include analyzing the query execution plan, identifying parallelizable steps, determining the number of processing units required, splitting the query into sub-tasks, allocating processing units, executing sub-tasks in parallel, and monitoring performance. Parallelism can be implemented using various techniques, such as shared-memory parallelism, distributed-memory parallelism, and hybrid parallelism. The choice of parallelism technique will depend on the hardware resources available, the nature of the query, and the size of the dataset. Optimizing database queries using parallelism requires careful planning and testing to ensure optimal performance. It is important to monitor the system performance and make adjustments as necessary to avoid bottlenecks and inefficiencies. Overall, database query optimization using parallelism is a powerful technique that can help improve database performance and handle large datasets efficiently.

Dataset Description :

This mini project uses sqlite database that store programmatically generated data in 3 tables that follows similar schema (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)

Algorithm :

1. Analyze the query execution plan: The first step in optimizing database queries using parallelism is to analyze the query execution plan. The query execution plan identifies the steps involved in executing the query and the dependencies between them. The goal is to identify the parts of the query that can be executed in parallel without creating conflicts.
2. Identify parallelizable steps: Once we have analyzed the query execution plan, we need to identify the steps that can be executed in parallel. These steps should be independent of each other and should

not rely on the output of previous steps.

3. Determine the number of processing units: The number of processing units required for parallel execution will depend on the number of parallelizable steps and the hardware resources available.

It is important to ensure that the number of processing units is sufficient to avoid any bottlenecks.

4. Split the query into subtasks: The next step is to split the query into subtasks that can be executed in parallel. The subtasks should be independent of each other and should not rely on the output of previous subtasks.

5. Allocate processing units: Once we have split the query into subtasks, we need to allocate processing units to each subtask. The goal is to ensure that each processing unit is utilized efficiently and that there are no bottlenecks.

6. Execute subtasks in parallel: The final step is to execute the subtasks in parallel using the allocated processing units. The results of each subtask are combined to produce the final result. It is important to ensure that the results are combined correctly and that there are no data inconsistencies.

7. Monitor performance: Throughout the execution of the query, it is important to monitor the performance and make adjustments as necessary. This includes monitoring the processing time, the utilization of processing units, and the overall system performance. Any bottlenecks or inefficiencies should be identified and addressed to ensure optimal performance.

CODE ::

```
import multiprocessing
import sqlite3
N = 10
# Define the function that will execute the query
def execute_query(query):
    conn = sqlite3.connect('database.db')
    cursor = conn.cursor()
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result
conn = sqlite3.connect('database.db')
# Create a cursor object to execute SQL statements
cursor = conn.cursor()
# Define the SQL statement to create the table
for i in range(1,N+1):
```

```

create_table_query = f"""
CREATE TABLE IF NOT EXISTS table{i} (
id INTEGER PRIMARY KEY,
name TEXT,
age INTEGER
);

# Execute the SQL statement to create the table
cursor.execute(create_table_query)

insertQuery = f"insert into table{i} (id, name, age) values({i},
'num_{i}', {i**2});"
print(insertQuery)
cursor.execute(insertQuery)

query = f"select * from table{i}"
cursor.execute(query)
items = cursor.fetchall()
print(items)

# Commit the changes to the database
conn.commit()
conn.close()

# Define the queries to be executed in parallel
queries = [
f"SELECT * FROM table{i}" for i in range(1,N+1)
]

# Create a pool of processes to execute the queries
with multiprocessing.Pool(processes=len(queries)) as pool:
# Execute the queries in parallel and get the results
results = pool.map(execute_query, queries)

# Print the results
for result in results:
print(result)

```

Output :

```
● (env) niteesh@Niteesh:~/LP5Project$ /home/niteesh/LP5Project/env/bin/python /home/niteesh/LP5Project/Project1DBQuery/query0
ptimization.py
insert into table1 (id, name, age) values(1, "num_1", 1);
[(1, 'num_1', 1)]
insert into table2 (id, name, age) values(2, "num_2", 4);
[(2, 'num_2', 4)]
insert into table3 (id, name, age) values(3, "num_3", 9);
[(3, 'num_3', 9)]
insert into table4 (id, name, age) values(4, "num_4", 16);
[(4, 'num_4', 16)]
insert into table5 (id, name, age) values(5, "num_5", 25);
[(5, 'num_5', 25)]
insert into table6 (id, name, age) values(6, "num_6", 36);
[(6, 'num_6', 36)]
insert into table7 (id, name, age) values(7, "num_7", 49);
[(7, 'num_7', 49)]
insert into table8 (id, name, age) values(8, "num_8", 64);
[(8, 'num_8', 64)]
insert into table9 (id, name, age) values(9, "num_9", 81);
[(9, 'num_9', 81)]
insert into table10 (id, name, age) values(10, "num_10", 100);
[(10, 'num_10', 100)]
[(1, 'num_1', 1)]
[(2, 'num_2', 4)]
[(3, 'num_3', 9)]
[(4, 'num_4', 16)]
[(5, 'num_5', 25)]
[(6, 'num_6', 36)]
[(7, 'num_7', 49)]
[(8, 'num_8', 64)]
[(9, 'num_9', 81)]
[(10, 'num_10', 100)]
```

Conclusion: In this way we can demonstrated that parallelism can significantly improve database query performance. By implementing parallelism in queries, we can achieve faster response times and reduce the overall processing time. However, it is essential to identify the parts of the query that can be executed in parallel to avoid any overheads that might reduce the benefits of parallelism.

Part II Deep Learning

Experiment No: - 1

Title: Linear regression by using Deep Neural network for Boston housing price prediction.

Problem Statement: To Implement Boston housing price prediction problem by linear regression using Deep Neural network. Use Boston House price prediction dataset.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

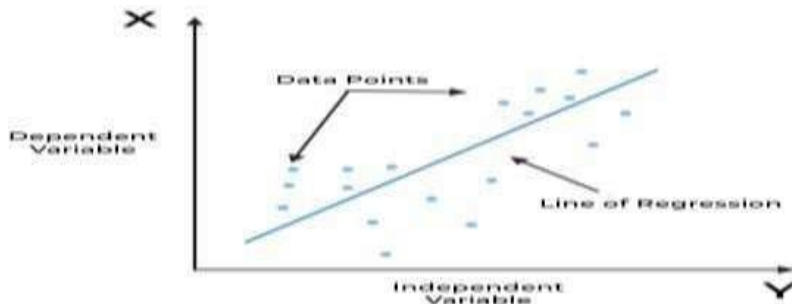
Software:

- Python programming language
- Keras for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory:

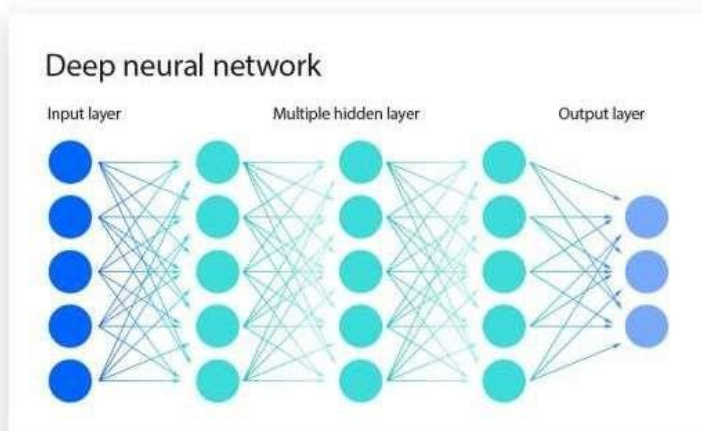
What is Linear Regression?

Linear regression is a statistical approach that is commonly used to model the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship between the variables and uses mathematical methods to estimate the coefficients that best fit the data. Deep neural networks are a type of machine learning algorithm that are modeled after the structure and function of the human brain. They consist of multiple layers of interconnected neurons that process data and learn from it to make predictions or classifications. Linear regression using deep neural networks combines the principles of linear regression with the power of deep learning algorithms. In this approach, the input features are passed through one or more layers of neurons to extract features and then a linear regression model is applied to the output of the last layer to make predictions. The weights and biases of the neural network are adjusted during training to optimize the performance of the model. This approach can be used for a variety of tasks, including predicting numerical values, such as stock prices or housing prices, and classifying data into categories, such as detecting whether an image contains a particular object or not. It is often used in fields such as finance, healthcare, and image recognition.



Concept of Deep Neural Network

A deep neural network is a type of machine learning algorithm that is modeled after the structure and function of the human brain. It consists of multiple layers of interconnected nodes, or artificial neurons, that process data and learn from it to make predictions or classifications. Each layer of the network performs a specific type of processing on the data, such as identifying patterns or correlations between features, and passes the results to the next layer. The layers closest to the input are known as the "input layer", while the layers closest to the output are known as the "output layer". The intermediate layers between the input and output layers are known as "hidden layers". These layers are responsible for extracting increasingly complex features from the input data, and can be deep (i.e., containing many hidden layers) or shallow (i.e., containing only a few hidden layers). Deep neural networks are trained using a process known as back propagation, which involves adjusting the weights and biases of the nodes based on the error between the predicted output and the actual output. This process is repeated for multiple iterations until the model reaches an optimal level of accuracy. Deep neural networks are used in a variety of applications, such as image and speech recognition, natural language processing, and recommendation systems. They are capable of learning from vast amounts of data and can automatically extract features from raw data, making them a powerful tool for solving complex problems in a wide range of domains.



How Deep Neural Network Work ?

Boston House Price Prediction is a common example used to illustrate how a deep neural network can work for regression tasks. The goal of this task is to predict the price of a house in Boston based on

various features such as the number of rooms, crime rate, and accessibility to public transportation. Here's how a deep neural network can work for Boston House Price Prediction:

1. **Data preprocessing:** The first step is to preprocess the data. This involves normalizing the input features to have a mean of 0 and a standard deviation of 1, which helps the network learn more efficiently. The dataset is then split into training and testing sets.
2. **Model architecture:** A deep neural network is then defined with multiple layers. The first layer is the input layer, which takes in the normalized features. This is followed by several hidden layers, which can be deep or shallow. The last layer is the output layer, which predicts the house price.
3. **Model training:** The model is then trained using the training set. During training, the weights and biases of the nodes are adjusted based on the error between the predicted output and the actual output. This is done using an optimization algorithm such as stochastic gradient descent.
4. **Model evaluation:** Once the model is trained, it is evaluated using the testing set. The performance of the model is measured using metrics such as mean squared error or mean absolute error.
5. **Model prediction:** Finally, the trained model can be used to make predictions on new data, such as predicting the price of a new house in Boston based on its features.
6. By using a deep neural network for Boston House Price Prediction, we can obtain accurate predictions based on a large set of input features. This approach is scalable and can be used for other regression tasks as well.

Algorithm:

1. Data Loading and Preprocessing:

- Load the dataset using Pandas.
- Handle missing values by replacing them with the mean of each respective column.
- Separate the features (X) and the target variable (Y).
- Split the dataset into training and testing sets.

2. Feature Scaling:

- Standardize the features to have a mean of 0 and a standard deviation of 1 using StandardScaler from scikit-learn.

3. Model Creation:

- Create a sequential model using Keras.
- Add layers to the model:
 - Input layer with 128 neurons, using ReLU activation.

- Hidden layer with 64 neurons, also using ReLU activation.
- Output layer with 1 neuron for regression, using linear activation.
- Compile the model with the Adam optimizer, mean squared error (MSE) loss function, and mean absolute error (MAE) as the evaluation metric.

4. Model Training:

- Fit the model to the training data for 100 epochs with a validation split of 0.05.

5. Model Evaluation:

- Evaluate the model performance on the test set using the evaluate method, which returns the MSE and MA

Code :

```
import pandas as pd
import numpy as np

df = pd.read_csv('HousingData.csv')
df.head()
df.isnull().sum()

crim_avg = df['CRIM'].astype('float').mean()
df['CRIM'].replace(np.NaN, crim_avg, inplace=True)
zn_avg = df['ZN'].astype('float').mean()
df['ZN'].replace(np.NaN, zn_avg, inplace=True)
indus_avg = df['INDUS'].astype('float').mean()
df['INDUS'].replace(np.NaN, indus_avg, inplace=True)
chas_avg = df['CHAS'].astype('float').mean()
df['CHAS'].replace(np.NaN, chas_avg, inplace=True)
age_avg = df['AGE'].astype('float').mean()
df['AGE'].replace(np.NaN, age_avg, inplace=True)
lstat_avg = df['LSTAT'].astype('float').mean()
df['LSTAT'].replace(np.NaN, lstat_avg, inplace=True)

df.corr()['MEDV'].sort_values()

X = df.loc[:,['LSTAT','PTRATIO','RM']]
Y = df.loc[:, "MEDV"]
X.shape,Y.shape

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(X,Y,test_size=0.25,random_state=10)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(x_train)
x_train = scaler.transform(x_train)
```

```
x_test = scaler.transform(x_test)
```

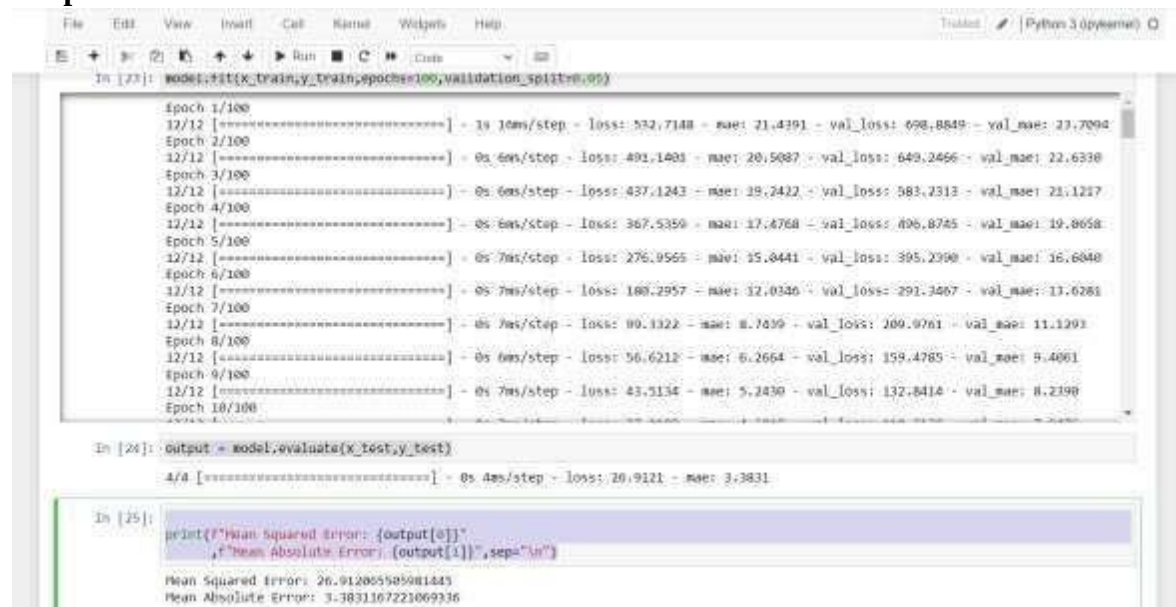
```
from keras.models import Sequential
from keras.layers import Dense
model = Sequential()
model.add(Dense(128,input_shape=(3,),activation='relu',name='input'))
model.add(Dense(64,activation='relu',name='layer_1'))
model.add(Dense(1,activation='linear',name='output'))
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

```
model.fit(x_train,y_train,epochs=100,validation_split=0.05)
```

```
output = model.evaluate(x_test,y_test)
```

```
print(f"Mean Squared Error: {output[0]}",f"Mean Absolute Error: {output[1]}",sep="\n")
```

Output :



```
In [23]: model.fit(x_train,y_train,epochs=100,validation_split=0.05)

Epoch 1/100
12/12 [=====] - 1s 10ms/step - loss: 532.7148 - mae: 21.4391 - val_loss: 698.8849 - val_mae: 23.7094
Epoch 2/100
12/12 [=====] - 0s 6ms/step - loss: 491.1403 - mae: 20.5087 - val_loss: 649.2466 - val_mae: 22.6330
Epoch 3/100
12/12 [=====] - 0s 6ms/step - loss: 437.1243 - mae: 18.7422 - val_loss: 583.2313 - val_mae: 21.1217
Epoch 4/100
12/12 [=====] - 0s 6ms/step - loss: 367.5359 - mae: 17.4768 - val_loss: 496.8745 - val_mae: 19.8658
Epoch 5/100
12/12 [=====] - 0s 7ms/step - loss: 276.9565 - mae: 15.8441 - val_loss: 395.2990 - val_mae: 16.6840
Epoch 6/100
12/12 [=====] - 0s 7ms/step - loss: 188.2957 - mae: 12.0346 - val_loss: 291.3467 - val_mae: 13.6281
Epoch 7/100
12/12 [=====] - 0s 7ms/step - loss: 99.1322 - mae: 8.7439 - val_loss: 169.9761 - val_mae: 11.1293
Epoch 8/100
12/12 [=====] - 0s 6ms/step - loss: 56.6212 - mae: 6.2664 - val_loss: 159.4785 - val_mae: 9.4061
Epoch 9/100
12/12 [=====] - 0s 7ms/step - loss: 43.5134 - mae: 5.2430 - val_loss: 132.8414 - val_mae: 8.2390
Epoch 10/100

In [24]: output = model.evaluate(x_test,y_test)

4/4 [=====] - 0s 4ms/step - loss: 26.9121 - mae: 3.3831

In [25]: print(f"Mean Squared Error: {output[0]}",
              f"Mean Absolute Error: {output[1]}",sep="\n")

Mean Squared Error: 26.912065585981443
Mean Absolute Error: 3.3831167221069336
```

Conclusion: In this way we can predict the Boston House Price using Deep Neural Network.

Experiment No :- 2(a)

Title: Binary Classification of Movie Reviews Using Deep Neural Networks.

Problem Statement: Binary classification using Deep Neural Networks Example: Classify movie reviews into "positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset.

Hardware and Software Requirements:

A machine with GPU support (recommended) for faster training times.
Sufficient RAM and disk space for dataset storage and model training.

Software:

Python programming language
TensorFlow or PyTorch library for deep learning
Pandas and NumPy for data manipulation
Matplotlib or Seaborn for data visualization

Theory:

What is Classification?

Classification is a type of supervised learning in machine learning that involves categorizing data into predefined classes or categories based on a set of features or characteristics. It is used to predict the class of new, unseen data based on the patterns learned from the labeled training data. In classification, a model is trained on a labeled dataset, where each data point has a known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data.

Classification is a common task in deep neural networks, where the goal is to predict the class of an input based on its features. Here's an example of how classification can be performed in a deep neural network using the popular MNIST dataset of handwritten digits. The MNIST dataset contains 60,000 training images and 10,000 testing images of handwritten digits from 0 to 9. Each image is a grayscale 28x28 pixel image, and the task is to classify each image into one of the 10 classes corresponding to the 10 digits. We can use a convolutional neural network (CNN) to classify the MNIST dataset. A CNN is a type of deep neural network that is commonly used for image classification tasks.

How Deep Neural Network Work on Classification:

Deep neural networks are commonly used for classification tasks because they can automatically learn to extract relevant features from raw input data and map them to the correct output class.

The basic architecture of a deep neural network for classification consists of three main parts: an input layer, one or more hidden layers, and an output layer. The input layer receives the raw input data, which is usually preprocessed to a fixed size and format. The hidden layers are composed of neurons that apply linear transformations and nonlinear activations to the input features to extract relevant patterns and representations. Finally, the output layer produces the predicted class labels, usually as a probability distribution over the possible classes.

One of the key advantages of deep neural networks for classification is their ability to learn hierarchical

representations of the input data. In a deep neural network with multiple hidden layers, each layer learns to capture more complex and abstract features than the previous layer, by building on the representations learned by the earlier layers. This hierarchical structure allows deep neural networks to learn highly discriminative features that can separate different classes of input data, even when the data is highly complex or noisy.

IMDB Dataset-The IMDB dataset is a large collection of movie reviews collected from the IMDB website, which is a popular source of user-generated movie ratings and reviews. The dataset consists of 50,000 movie reviews, split into 25,000 reviews for training and 25,000 reviews for testing. Each review is represented as a sequence of words, where each word is represented by an integer index based on its frequency in the dataset. The labels for each review are binary, with 0 indicating a negative review and 1 indicating a positive review.

Deep learning approaches, such as deep neural networks, have achieved state-of-the-art performance on the IMDB dataset by automatically learning to extract relevant features from the raw text data and map them to the correct output class. The IMDB dataset is widely used in research and education for natural language processing and machine learning, as it provides a rich source of labeled text data for training and testing deep learning models.

Algorithm:

Data Preprocessing: Load the IMDB dataset, preprocess the text data (tokenization, padding), and split it into training and testing sets.

Model Architecture: Design a deep neural network architecture suitable for text classification. This typically involves embedding layers followed by one or more layers of LSTM or CNN units, and finally, dense layers for classification.

Model Compilation: Compile the model with an appropriate loss function (e.g., binary cross-entropy) and optimizer (e.g., Adam).

Model Training: Train the model on the training data, monitoring performance on the validation set to avoid overfitting.

Model Evaluation: Evaluate the trained model on the testing set to assess its performance in classifying movie reviews.

Prediction: Use the trained model to make predictions on new, unseen data.

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from keras.datasets import imdb
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
data = np.concatenate((X_train, X_test), axis=0)
label = np.concatenate((y_train, y_test), axis=0)

print("Review is ",X_train[0]) # series of no converted word to vocabulary associated with index
print("Review is ",y_train[0])
vocab=imdb.get_word_index() # Retrieve the word index file mapping words to indices
print(vocab)

def vectorize(sequences, dimension = 10000): # We will vectorize every review and fill it with zeros
```



```

# so that it contains exactly 10,000 numbers.
# Create an all-zero matrix of shape (len(sequences), dimension)
results = np.zeros((len(sequences), dimension))
for i, sequence in enumerate(sequences):
    results[i, sequence] = 1
return results

test_x = data[:10000]
test_y = label[:10000]
train_x = data[10000:]
train_y = label[10000:]
test_x.shape
print("Categories:", np.unique(label))
print("Number of unique words:", len(np.unique(np.hstack(data))))

length = [len(i) for i in data]
print("Average Review length:", np.mean(length))
print("Standard Deviation:", round(np.std(length)))

print("Label:", label[0])
Label: 1
print("Label:", label[1])
Label: 0
print(data[0])

index = imdb.get_word_index()
reverse_index = dict([(value, key) for (key, value) in index.items()]) # id to word
decoded = " ".join([reverse_index.get(i - 3, "#") for i in data[0]])
print(decoded)

import seaborn as sns

data = vectorize(data)
label = np.array(label).astype("float32")
labelDF=pd.DataFrame({'label':label})
sns.countplot(x='label', data=labelDF)

<Axes: xlabel='label', ylabel='count'
# Creating train and test data set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(data,label, test_size=0.20, random_state=1)
X_train.shape
(40000, 10000)
X_test.shape
(10000, 10000)
# Let's create sequential model
from keras.utils import to_categorical
from keras import models

```

```

from keras import layers
model = models.Sequential()
Model: "sequential_2"

```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 50)	500050
dropout_2 (Dropout)	(None, 50)	0
dense_5 (Dense)	(None, 50)	2550
dropout_3 (Dropout)	(None, 50)	0
dense_6 (Dense)	(None, 50)	2550
dense_7 (Dense)	(None, 1)	51

```

Total params: 505201 (1.93 MB)
Trainable params: 505201 (1.93 MB)
Non-trainable params: 0 (0.00 Byte)

```

```

model.add(layers.Dense(50, activation = "relu", input_shape=(10000, )))
model.add(layers.Dropout(0.3, noise_shape=None, seed=None))
model.add(layers.Dense(50, activation = "relu"))
model.add(layers.Dropout(0.2, noise_shape=None, seed=None))
model.add(layers.Dense(50, activation = "relu"))
# Output- Layer
model.add(layers.Dense(1, activation = "sigmoid"))
model.summary()

```

Conclusion:

Using the IMDB dataset and deep learning techniques, we created a robust neural network model for sentiment analysis of movie reviews. This highlights the power of deep learning in natural language processing applications.

```

# Import necessary libraries import numpy as np import pandas as pd
import matplotlib.pyplot as plt import tensorflow as tf
from sklearn.model_selection import train_test_split from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
In [2]:
In [3]: # Load the OCR letter recognition dataset data_url = "https://archive.ics.uci.edu/ml/machine-
learning-databases/letter-recogn column_names = ['letter', 'x-box', 'y-box', 'width', 'height', 'total_pixels', 'x-
b df = pd.read_csv(data_url, names=column_names)
In [4]: # Display dataset information print("Dataset Information:") print(df.info()) print("\nSample of the
Dataset:") print(df.head())
Dataset Information:
<class 'pandas.core.frame.DataFrame'>

```

RangeIndex: 20000 entries, 0 to 19999 Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype	--- -----	-----	-----	-----	0	letter	20000 non-null	
object											
1	x-box	20000 non-null	int64								
2	y-box	20000 non-null	int64	3	width		20000 non-null	int64			
4	height	20000 non-null	int64								
5	total_pixels	20000 non-null	int64								
6	x-bar	20000 non-null	int64	7	y-bar		20000 non-null	int64			
8	x2-bar	20000 non-null	int64								
9	y2-bar	20000 non-null	int64								
10	xy-bar	20000 non-null	int64	11	x2y-bar		20000 non-null	int64			
12	xy2-bar	20000 non-null	int64								
13	x-ege	20000 non-null	int64								
14	xegvy	20000 non-null	int64	15	y-ege		20000 non-null	int64	16	yegvx	20000 non-null

int64 dtypes: int64(16), object(1) memory usage: 2.6+ MB None

Sample of the Dataset:

	letter	x-box	y-box	width	height	total_pixels	x-bar	y-bar	x2-bar	\ 0	T	2	8	3	5	
1	8	13	0													
1	I	5	12	3	7	2	10	5	5							
2	D	4	11	6	8	6	10	6	2							
3	N	7	11	6	6	3	5	9	4							
4	G	2	1	3	1	1	8	6	6							
	y2-bar	xy-bar	x2y-bar	xy2-bar	x-ege	xegvy	y-ege	yegvx	0	6	6	10	8	0	8	0
8	1	4	13	3	9	2	8	4	10							
2	6	10	3	7	3	7	3	9								
3	6	4	4	10	6	10	2	8								
4	6	6	5	9	1	7	5	10								

Preprocess the data

```
X = df.drop('letter', axis=1) y = df['letter'] le = LabelEncoder() y = le.fit_transform(y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [5]:

```
In [6]: # Define the DNN model architecture model = tf.keras.Sequential([ tf.keras.layers.Dense(128,
activation='relu', input_shape=(16,)),
tf.keras.layers.Dense(64, activation='relu'),
tf.keras.layers.Dense(26, activation='softmax') ])
```

C:\Users\rsku\anaconda3\lib\site-packages\keras\src\layers\core\dense.py:85: User Warning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [7]: # Compile the model model.compile(optimizer='adam',
```

```
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
In [8]: # Train the model history = model.fit(X_train, y_train, epochs=10, batch_size=64,
```

```
validation_split=0.
```

Epoch 1/10

200/200 ————— 1s 2ms/step - accuracy: 0.2179 - loss: 2.9413 -

val_accuracy: 0.6197 - val_loss: 1.4544

Epoch 2/10

200/200 ————— 0s 2ms/step - accuracy: 0.6364 - loss: 1.3274 -

val_accuracy: 0.7184 - val_loss: 1.0582

```

Epoch 3/10
200/200 ————— 0s 2ms/step - accuracy: 0.7245 - loss: 0.9896 -
val_a ccuracy: 0.7575 - val_loss: 0.9138 Epoch 4/10
200/200 ————— 0s 1ms/step - accuracy: 0.7689 - loss: 0.8156 -
val_a ccuracy: 0.7653 - val_loss: 0.8406
Epoch 5/10
200/200 ————— 0s 1ms/step - accuracy: 0.7918 - loss: 0.7458 -
val_a ccuracy: 0.8084 - val_loss: 0.7327
Epoch 6/10
200/200 ————— 0s 2ms/step - accuracy: 0.8082 - loss: 0.6837 -
val_a ccuracy: 0.8159 - val_loss: 0.6627
Epoch 7/10
200/200 ————— 0s 2ms/step - accuracy: 0.8182 - loss: 0.6401 -
val_a ccuracy: 0.8350 - val_loss: 0.6150 Epoch 8/10
200/200 ————— 0s 1ms/step - accuracy: 0.8340 - loss: 0.5844 -
val_a ccuracy: 0.8322 - val_loss: 0.5907
Epoch 9/10
200/200 ————— 0s 2ms/step - accuracy: 0.8419 - loss: 0.5494 -
val_a ccuracy: 0.8391 - val_loss: 0.5586
Epoch 10/10
200/200 ————— 0s 1ms/step - accuracy: 0.8518 - loss: 0.5147 -
val_a ccuracy: 0.8431 - val_loss: 0.5273
In [9]: # Evaluate the model test_loss, test_acc = model.evaluate(X_test, y_test) print('Test accuracy:',
test_acc)
125/125 ————— 0s 873us/step - accuracy: 0.8566 - loss: 0.4996
Test accuracy: 0.856249988079071
In [10]: # Make predictions predictions = model.predict(X_test) y_pred = np.argmax(predictions, axis=1)
125/125 ————— 0s 897us/step
# Display some sample predictions
sample_indices = np.random.choice(len(X_test), 10, replace=False) print("\nSample Predictions:") for i,
index in enumerate(sample_indices):
    true_label = le.inverse_transform([y_test[index]])[0]
    predicted_label =
le.inverse_transform([y_pred[index]])[0] print(f'Sample {i+1}: True Label: {true_label}, Predicted
Label: {predicted_label}')
In [11]:
Sample Predictions:
Sample 1: True Label: T, Predicted Label: T Sample 2: True Label: E, Predicted Label: E
Sample 3: True Label: I, Predicted Label: L
Sample 4: True Label: V, Predicted Label: V
Sample 5: True Label: I, Predicted Label: I Sample 6: True Label: Z, Predicted Label: Z
Sample 7: True Label: Y, Predicted Label: Y
Sample 8: True Label: Z, Predicted Label: Z
Sample 9: True Label: C, Predicted Label: C
Sample 10: True Label: S, Predicted Label: S
In [12]: # Classification report print("\nClassification Report:") print(classification_report(y_test, y_pred))
Classification Report:
              precision    recall  f1-score   support
0               0.82      0.92      0.86         149

```


Experiment No :- 2(b)

Title: Multiclass Classification using Deep Neural Networks

Problem Statement:

Develop a deep neural network (DNN) for multiclass classification using the OCR letter recognition dataset. The goal is to accurately classify letters into their respective categories based on input features.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

Software:

- Python programming language
- TensorFlow framework for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory:

Deep Neural Networks (DNNs) are a class of artificial neural networks characterized by multiple hidden layers between the input and output layers. They are capable of learning complex representations from raw data, enabling them to solve a wide range of machine learning tasks, including classification, regression, and feature learning.

Key Components of Deep Neural Networks:

1. Neurons:

Neurons are the basic computational units of DNNs. Each neuron receives inputs, performs a weighted sum of those inputs, adds a bias term, and applies an activation function to produce an output.

2. Layers:

DNNs consist of multiple layers, including input, hidden, and output layers. Each layer is composed of neurons connected to neurons in adjacent layers.

The input layer receives raw data features, the hidden layers process and transform the features, and the output layer produces the final predictions or classifications.

3. Activation Functions:

Activation functions introduce non-linearity into the network, allowing DNNs to model complex relationships in the data.

Common activation functions include ReLU (Rectified Linear Unit), sigmoid, tanh (hyperbolic tangent), and softmax.

4. Weights and Biases:

Weights represent the strength of connections between neurons in adjacent layers. They are adjusted during training to minimize the difference between predicted and actual outputs.

Biases are additional parameters added to each neuron that allow the network to learn even when all inputs are zero.

5. Backpropagation:

Backpropagation is the training algorithm used to update the weights and biases of a DNN based on the difference between predicted and actual outputs.

It involves calculating the gradient of the loss function with respect to the network parameters and using

this gradient to update the parameters in the direction that minimizes the loss.

DNN Architecture:

Input Layer: Receives raw data features. In this OCR letter recognition dataset, the input layer has 16 neurons corresponding to the features of each letter image.

Hidden Layers: Process and transform the input features through weighted connections and activation functions. The architecture includes multiple hidden layers with varying numbers of neurons, allowing the network to learn hierarchical representations of the data.

Output Layer: Produces the final predictions or classifications. In this multiclass classification task, the output layer has 26 neurons, each representing a letter category. The softmax activation function is applied to produce probability distributions over the classes.

Training Process:

- DNNs are trained using labelled data through a process called backpropagation. During training, the model adjusts its internal parameters (weights and biases) to minimize a predefined loss function.
- Gradient descent optimization algorithms, such as Adam or stochastic gradient descent (SGD), are commonly used to update the model parameters iteratively.
- Training is typically performed on a training dataset, with a portion of the data reserved for validation to monitor the model's performance and prevent overfitting.

Evaluation Metrics:

- Various metrics can be used to evaluate the performance of a classification model, including accuracy, precision, recall, F1-score, and confusion matrix.
- Accuracy measures the proportion of correctly classified instances out of the total number of instances.
- Precision represents the proportion of true positive predictions among all positive predictions.
- Recall (or sensitivity) measures the proportion of true positive predictions among all actual positive instances.
- F1-score is the harmonic mean of precision and recall, providing a balance between the two metrics.
- Confusion matrix visualizes the performance of the classifier by showing the counts of true positive, true negative, false positive, and false negative predictions.

Code/Output:

```
# Import necessary libraries import numpy as np import pandas as pd import matplotlib.pyplot as plt
import tensorflow as tf from sklearn.model_selection import train_test_split from sklearn.preprocessing
import LabelEncoder from sklearn.metrics import classification_report

# Load the OCR letter recognition dataset
data_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/letter-
recognition/letterrecognition.data"
column_names = ['letter', 'x-box', 'y-box', 'width', 'height', 'total_pixels', 'x-bar', 'y-bar', 'x2-bar', 'y2bar',
'xy-bar', 'x2y-bar', 'xy2-bar', 'x-ege', 'xegvy', 'y-ege', 'yegvx'] df = pd.read_csv(data_url,
names=column_names)

# Display dataset information print("Dataset Information:")
print(df.info()) print("\nSample of the Dataset:") print(df.head())

# Preprocess the data
```

```

X = df.drop('letter', axis=1) y = df['letter']
le = LabelEncoder() y = le.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the DNN model architecture model = tf.keras.Sequential([
    tf.keras.layers.Dense(128,
activation='relu', input_shape=(16,)),
    tf.keras.layers.Dense(64, activation='relu'),
tf.keras.layers.Dense(26, activation='softmax')
])

# Compile the model model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

# Evaluate the model test_loss, test_acc = model.evaluate(X_test, y_test) print('Test accuracy:', test_acc)

# Make predictions predictions = model.predict(X_test) y_pred = np.argmax(predictions, axis=1) #
Display some sample predictions sample_indices = np.random.choice(len(X_test), 10, replace=False)
print("\nSample Predictions:") for i, index in enumerate(sample_indices):
    true_label = le.inverse_transform([y_test[index]])[0] predicted_label =
le.inverse_transform([y_pred[index]])[0] print(f"Sample {i+1}: True Label: {true_label}, Predicted
Label: {predicted_label}")

# Classification report print("\nClassification Report:") print(classification_report(y_test, y_pred))

```

Conclusion:

In this practical assignment, we implemented a deep neural network (DNN) classifier for multiclass classification using the OCR letter recognition dataset. By training and evaluating the model, we demonstrated the effectiveness of DNNs in accurately classifying letters into their respective categories. Additionally, we discussed the underlying theory of DNNs, including their architecture, training process, and evaluation metrics. This assignment highlights the importance of deep learning techniques in handling complex classification tasks.

```

# Import necessary libraries import numpy as np import pandas as pd
import matplotlib.pyplot as plt import tensorflow as tf
from sklearn.model_selection import train_test_split from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report

In [2]:
In [3]: # Load the OCR letter recognition dataset data_url = "https://archive.ics.uci.edu/ml/machine-
learning-databases/letter-recogn column_names = ['letter', 'x-box', 'y-box', 'width', 'height', 'total_pixels', 'x-
b df = pd.read_csv(data_url, names=column_names)

In [4]: # Display dataset information print("Dataset Information:") print(df.info()) print("\nSample of the
Dataset:") print(df.head())

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999 Data columns (total 17 columns):
# Column Non-Null Count Dtype ---
letter 20000 non-null object

```



```

3      x-box      20000 non-null int64
4      y-box      20000 non-null int64 3 width      20000 non-null int64
7      height     20000 non-null int64
8      total_pixels 20000 non-null int64
9      x-bar      20000 non-null int64 7 y-bar      20000 non-null int64
11     x2-bar     20000 non-null int64
12     y2-bar     20000 non-null int64
13     xy-bar     20000 non-null int64 11 x2y-bar    20000 non-null int64
15     xy2-bar    20000 non-null int64
16     x-eg     20000 non-null int64
17     xegvy     20000 non-null int64 15 y-eg     20000 non-null int64 16 yegvx     20000 non-
null int64 dtypes: int64(16), object(1) memory usage: 2.6+ MB None

```

Sample of the Dataset:

```

letter x-box y-box width height total_pixels x-bar y-bar x2-bar \0      T      2      8      3      5
1      8      13      0
5      I      5      12      3      7      2      10      5      5
6      D      4      11      6      8      6      10      6      2
7      N      7      11      6      6      3      5      9      4
8      G      2      1      3      1      1      8      6      6
y2-bar xy-bar x2y-bar xy2-bar x-eg     xegvy y-eg     yegvx 0      6      6      10      8      0      8      0
8 1      4      13      3      9      2      8      4      10
5      6      10      3      7      3      7      3      9
6      6      4      4      10      6      10      2      8
7      6      6      5      9      1      7      5      10

```

Preprocess the data

```
X = df.drop('letter', axis=1) y = df['letter'] le = LabelEncoder() y = le.fit_transform(y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [5]:

```

In [6]: # Define the DNN model architecture model = tf.keras.Sequential([ tf.keras.layers.Dense(128,
activation='relu', input_shape=(16,)), tf.keras.layers.Dense(64, activation='relu'),
tf.keras.layers.Dense(26, activation='softmax') ])

```

C:\Users\rsuku\anaconda3\lib\site-packages\keras\src\layers\core\dense.py:85: User Warning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

In [7]: # Compile the model model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

```

```

In [8]: # Train the model history = model.fit(X_train, y_train, epochs=10, batch_size=64,
validation_split=0.

```

Epoch 1/10

```

200/200 ----- 1s 2ms/step - accuracy: 0.2179 - loss: 2.9413 -
val_a ccuracy: 0.6197 - val_loss: 1.4544

```

Epoch 2/10

```

200/200 ----- 0s 2ms/step - accuracy: 0.6364 - loss: 1.3274 -
val_a ccuracy: 0.7184 - val_loss: 1.0582

```

Epoch 3/10

```

200/200 ----- 0s 2ms/step - accuracy: 0.7245 - loss: 0.9896 -
val_a ccuracy: 0.7575 - val_loss: 0.9138 Epoch 4/10

```

```

200/200 _____ 0s 1ms/step - accuracy: 0.7689 - loss: 0.8156 -
val_a ccuracy: 0.7653 - val_loss: 0.8406
Epoch 5/10
200/200 _____ 0s 1ms/step - accuracy: 0.7918 - loss: 0.7458 -
val_a ccuracy: 0.8084 - val_loss: 0.7327
Epoch 6/10
200/200 _____ 0s 2ms/step - accuracy: 0.8082 - loss: 0.6837 -
val_a ccuracy: 0.8159 - val_loss: 0.6627
Epoch 7/10
200/200 _____ 0s 2ms/step - accuracy: 0.8182 - loss: 0.6401 -
val_a ccuracy: 0.8350 - val_loss: 0.6150 Epoch 8/10
200/200 _____ 0s 1ms/step - accuracy: 0.8340 - loss: 0.5844 -
val_a ccuracy: 0.8322 - val_loss: 0.5907
Epoch 9/10
200/200 _____ 0s 2ms/step - accuracy: 0.8419 - loss: 0.5494 -
val_a ccuracy: 0.8391 - val_loss: 0.5586
Epoch 10/10
200/200 _____ 0s 1ms/step - accuracy: 0.8518 - loss: 0.5147 -
val_a ccuracy: 0.8431 - val_loss: 0.5273
In [9]: # Evaluate the model test_loss, test_acc = model.evaluate(X_test, y_test) print("Test accuracy:",
test_acc)
125/125 _____ 0s 873us/step - accuracy: 0.8566 - loss: 0.4996
Test accuracy: 0.856249988079071
In [10]: # Make predictions predictions = model.predict(X_test) y_pred = np.argmax(predictions, axis=1)
125/125 _____ 0s 897us/step
# Display some sample predictions
sample_indices = np.random.choice(len(X_test), 10, replace=False) print("\nSample Predictions:") for i,
index in enumerate(sample_indices):
    true_label = le.inverse_transform([y_test[index]])[0] predicted_label =
le.inverse_transform([y_pred[index]])[0] print(f"Sample {i+1}: True Label: {true_label}, Predicted
Label: {predicted_lab}
In [11]:
Sample Predictions:
Sample 1: True Label: T, Predicted Label: T Sample 2: True Label: E, Predicted Label: E
Sample 3: True Label: I, Predicted Label: L
Sample 4: True Label: V, Predicted Label: V
Sample 5: True Label: I, Predicted Label: I Sample 6: True Label: Z, Predicted Label: Z
Sample 7: True Label: Y, Predicted Label: Y
Sample 8: True Label: Z, Predicted Label: Z
Sample 9: True Label: C, Predicted Label: C
Sample 10: True Label: S, Predicted Label: S
In [12]: # Classification report print("\nClassification Report:") print(classification_report(y_test, y_pred))
Classification Report:          precision recall f1-score support
3      0.82      0.92      0.86      149
4      0.73      0.86      0.79      153
5      0.92      0.80      0.86      137      3      0.90      0.82      0.86      156
7      0.70      0.88      0.78      141

```

[illegible]

Experiment No :- 3

Title: Convolutional Neural Network (CNN) for Fashion Clothing Classification.

Problem Statement: To develop a Convolutional Neural Network (CNN) classifier using the MNIST Fashion Dataset. The goal is to accurately classify fashion clothing items into predefined categories.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

Software:

- Python programming language
- TensorFlow or PyTorch framework for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory: Convolutional Neural Networks (CNNs) are a class of deep neural networks widely used for image recognition and classification tasks. They are particularly effective in handling spatial hierarchies of features through convolutional layers, pooling layers, and fully connected layers. CNNs have achieved state-of-the-art results in various computer vision tasks, including image classification, object detection, and segmentation.

CNNs consist of several layers, including convolutional layers, pooling layers, and fully connected layers. Convolutional layers use filters to extract features from input images through convolution operations. Pooling layers reduce spatial dimensions while preserving important information. Fully connected layers perform classification based on extracted features.

Activation functions like ReLU introduce non-linearity, allowing CNNs to learn complex mappings between features and classes. Dropout regularization prevents overfitting by randomly dropping out neurons during training.

CNNs are trained using gradient-based optimization algorithms like stochastic gradient descent. Transfer learning, where pre-trained CNN models are fine-tuned on specific tasks, accelerates training and improves performance, especially with limited training data.

In summary, CNNs are powerful tools for various computer vision tasks, providing state-of-the-art performance in image classification, object detection, and more.

Algorithm:

1. Data Preparation:

- Load the MNIST Fashion Dataset.
- Preprocess the data (e.g., normalize pixel values, reshape images).

2. CNN Model Architecture:

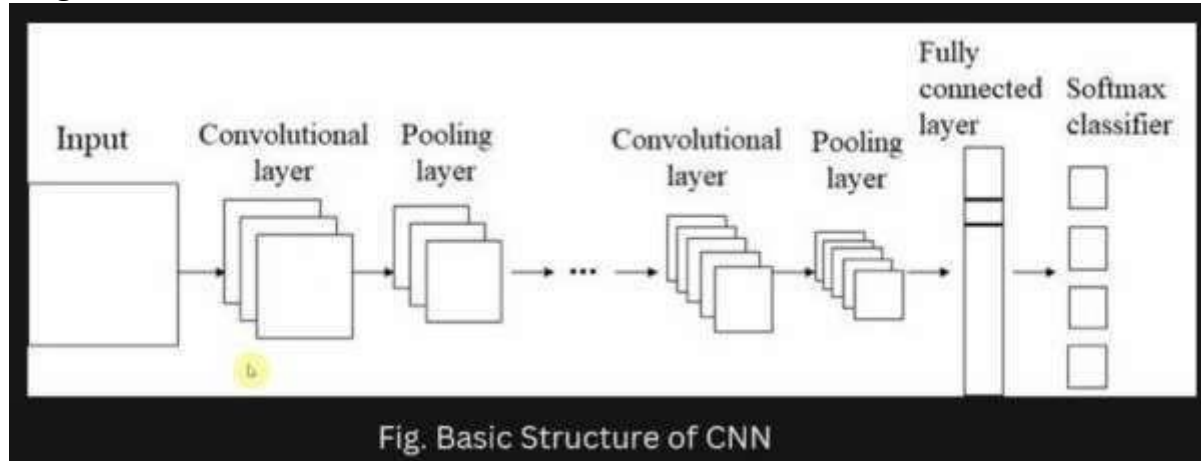
- Design a CNN architecture consisting of convolutional layers, pooling layers, and fully connected layers.
- Experiment with different configurations of layers, activation functions, and regularization techniques to optimize performance.

3. Training:

- Split the dataset into training, validation, and test sets.
- Train the CNN model using the training data.

- Validate the model's performance using the validation set and adjust hyperparameters accordingly.
4. Evaluation:
- Evaluate the trained model using the test set.
 - Calculate metrics such as accuracy, precision, recall, and F1-score to assess classification performance.
5. Prediction:
- Use the trained model to classify new fashion clothing images.

Diagram:



Code/Output:

Import necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
```

Load and preprocess the dataset

```
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)) / 255.0
test_images = test_images.reshape((10000, 28, 28, 1)) / 255.0
```

Define the CNN model architecture

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
```

Compile the model

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Train the model

```
history = model.fit(train_images, train_labels, epochs=10, batch_size=64, validation_split=0.2)
```

Evaluate the model

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('Test accuracy:', test_acc)
```

Plot training history

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.show()
```

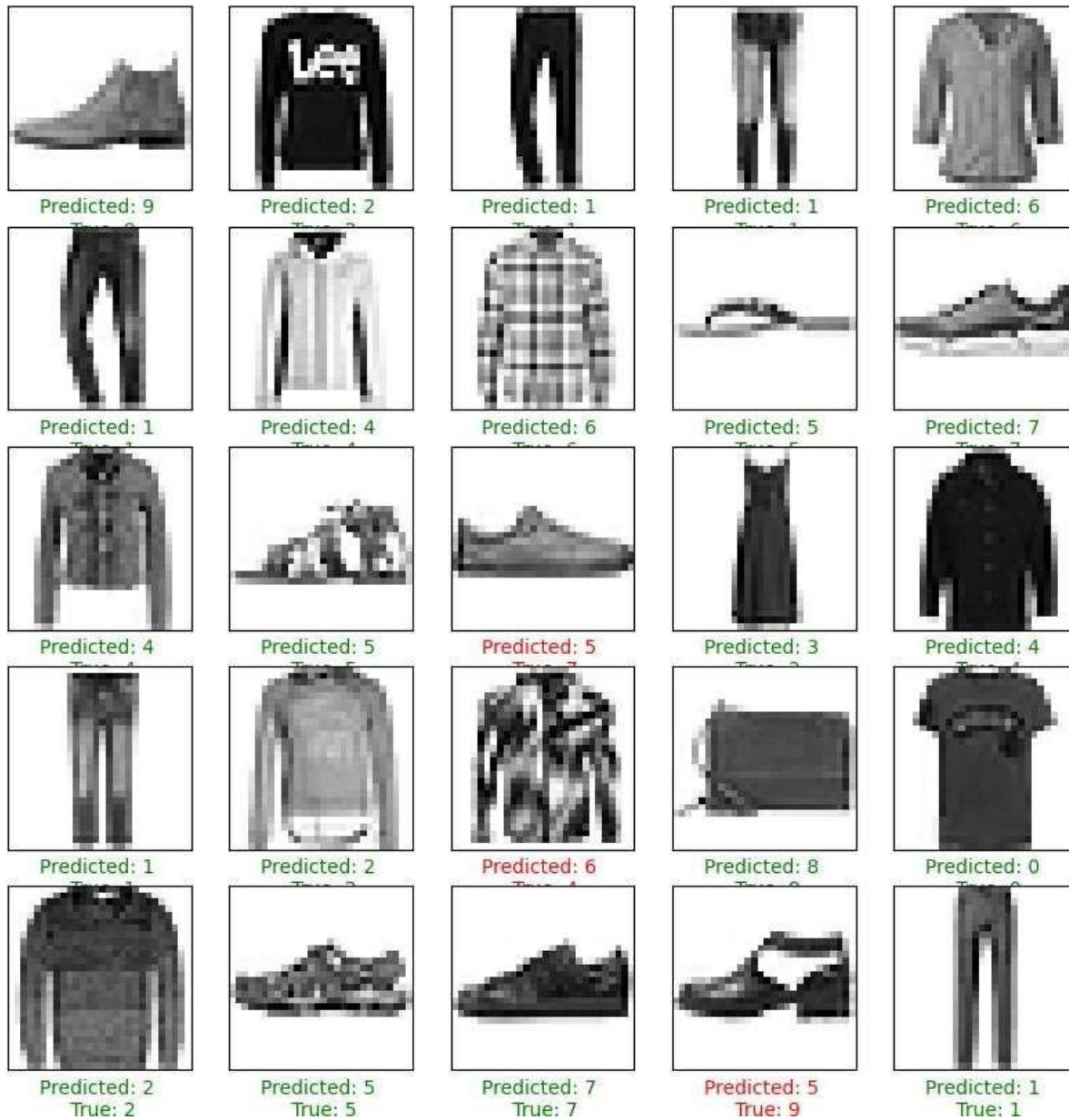
Make predictions

```
predictions = model.predict(test_images)
```

Display some sample predictions

```
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(test_images[i].reshape(28, 28), cmap=plt.cm.binary)
    predicted_label = np.argmax(predictions[i])
    true_label = test_labels[i]
    if predicted_label == true_label:
        color = 'green'
    else:
        color = 'red'
    plt.xlabel(f'Predicted: {predicted_label} \n True: {true_label}', color=color)
plt.show()
```

Output:



Conclusion: In this practical assignment, we developed a Convolutional Neural Network (CNN) classifier using the MNIST Fashion Dataset.

Experiment No:- 4

Title: Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

Problem Statement: Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

Software:

- Python programming language
- TensorFlow or PyTorch framework for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory:

Recurrent Neural Networks (RNNs) are a class of neural networks commonly used for sequential data processing tasks such as time series analysis, natural language processing (NLP), and speech recognition. They are effective in modeling sequential dependencies within data through feedback loops, enabling them to capture temporal dynamics.

RNNs consist of interconnected layers where each layer processes input sequences step by step. The key component of an RNN is the recurrent connection, which allows information to persist over time. This enables RNNs to maintain a memory of previous inputs and use it to influence future predictions.

In each time step, RNNs receive an input vector and a hidden state vector from the previous time step. These vectors are combined to produce an output vector and a new hidden state vector, which are then passed to the next time step. This recurrent process allows RNNs to capture sequential patterns and dependencies.

One challenge with traditional RNNs is the vanishing gradient problem, where gradients diminish exponentially over long sequences, leading to difficulty in learning long-term dependencies. To address

this, variants of RNNs like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures have been developed. These architectures incorporate mechanisms to selectively retain or forget information, allowing them to better capture long-term dependencies.

Activation functions such as tanh or ReLU are often used within RNN units to introduce non-linearity and enable the network to model complex relationships within sequential data.

RNNs are trained using gradient-based optimization algorithms like stochastic gradient descent (SGD) or its variants. Backpropagation through time (BPTT) is a common method for computing gradients in RNNs by unfolding the network over multiple time steps.

In summary, RNNs are powerful tools for processing sequential data, capable of capturing temporal dependencies and patterns. Variants like LSTM and GRU address challenges with learning long-term dependencies, making RNNs suitable for a wide range of tasks including NLP, time series analysis, and speech recognition.

Algorithm:

Data Preparation:

Obtain the Google stock prices dataset, ensuring it's preprocessed and cleaned. Split the dataset into training and testing sets. Typically, around 70-80% of the data is used for training and the rest for testing/validation.

Feature Selection/Engineering:

Decide on the features to be used for prediction. For stock price prediction, commonly used features include historical stock prices, volume, moving averages, etc. Normalize/standardize the features to ensure they're on a similar scale, which helps in training the neural network.

Model Architecture:

Choose an appropriate RNN architecture. Long Short-Term Memory (LSTM) networks are commonly used for time series prediction tasks due to their ability to capture long-term dependencies. Define the number of LSTM layers, number of units in each layer, and other hyperparameters. Consider adding dropout layers to prevent overfitting.

Model Compilation:

Compile the RNN model using an appropriate loss function (e.g., Mean Squared Error for regression tasks) and optimizer (e.g., Adam optimizer).

Model Training:

Train the RNN model using the training dataset. Monitor the model's performance on the validation set to avoid overfitting. Adjust hyperparameters if necessary.

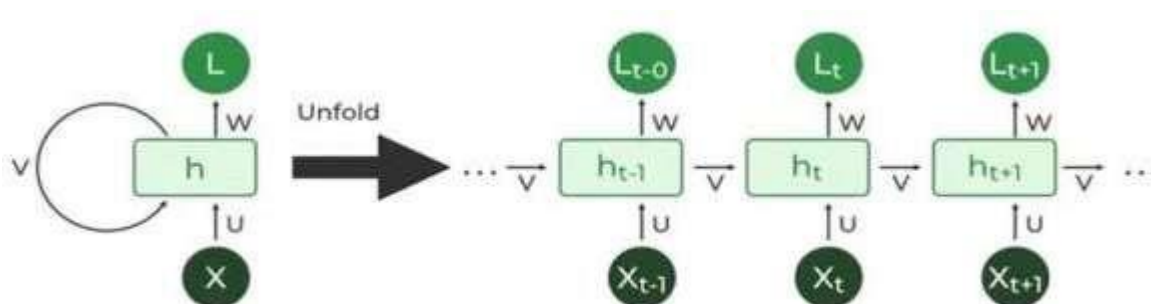
Model Evaluation:

Evaluate the trained model's performance on the testing dataset using appropriate evaluation metrics such

as Mean Absolute Error (MAE), Mean Squared Error (MSE), or Root Mean Squared Error (RMSE).

Prediction:

Utilize the trained model to make predictions on future stock prices. Optionally, visualize the predicted vs. actual prices to assess the model's accuracy visually.



Code/Output:

Import necessary libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import LSTM
```

```
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.layers import Dropout
```

Exploratory Data Analysis

```
plt.figure(figsize=(18, 8))
```

```
plt.plot(dataset_train['Open']) plt.title("Google
```

```
Stock Open Prices") plt.xlabel("Time (oldest ->
```

```
latest)") plt.ylabel("Stock Open Price")
```

```
plt.show()
```

Model Creation

```
regressor = Sequential()

regressor.add(LSTM(units = 50, return_sequences = True, input_shape = (X_train.shape[1], 1)))

regressor.add(Dropout(rate = 0.2))

regressor.add(LSTM(units = 50, return_sequences = True))

regressor.add(Dropout(rate = 0.2))

regressor.add(LSTM(units = 50, return_sequences = True))

regressor.add(Dropout(rate = 0.2))

regressor.add(LSTM(units = 50, return_sequences = False))

regressor.add(Dropout(rate = 0.2))

regressor.add(Dense(units = 1))
```

Compile the model

```
regressor.compile(optimizer = 'adam', loss = 'mean_squared_error')
```

Train the model

```
regressor.fit(x = X_train, y = y_train, batch_size = 32, epochs = 100)
```

Evaluate the model and visualize

```
predicted_stock_price = regressor.predict(X_test)

predicted_stock_price = sc.inverse_transform(predicted_stock_price)

plt.plot(real_stock_price, color = 'red', label = 'Real price')

plt.plot(predicted_stock_price, color = 'blue', label = 'Predicted price')

plt.title('Google price prediction') plt.xlabel('Time') plt.ylabel('Price')

plt.legend()

plt.show()
```

Conclusion: In this practical assignment, we developed a Recurrent Neural Network (RNN) using google stock prices dataset

Miniprojects

Title: Implement Human Face Recognition.

Project Title: To Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

Objective: To build a deep neural network model that can recognize the identities of celebrities in the "CelebA" dataset.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

Software:

- Python programming language
- Keras for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory:

Human face recognition using deep neural networks involves building a neural network model that can take an image of a human face as input and accurately recognize the person in the image. Here is a sample code using TensorFlow to implement a basic face recognition system using a deep neural network:

Human face recognition using deep neural networks (DNNs) involves training a neural network to identify and distinguish between different faces. The process typically involves the following steps:

Data collection: A large dataset of face images is collected, including images of different individuals and under different lighting and pose conditions.

Data preprocessing: The face images are preprocessed to remove noise, align the faces, and normalize the illumination.

Feature extraction: The preprocessed face images are then fed into a deep neural network to extract high-level features that capture the important characteristics of a face. The neural network typically consists of several layers of convolutional and pooling operations, followed by fully connected layers that produce a feature vector.

Training: The extracted features are then used to train the neural network to distinguish between different

faces. This is typically done using a supervised learning approach, where the network is trained on a labeled dataset of face images and their corresponding identities.

Testing: After the neural network has been trained, it can be tested on a separate dataset to evaluate its performance. This typically involves measuring the accuracy of the network in correctly identifying the individuals in the test dataset.

Deployment: Once the neural network has been trained and tested, it can be deployed in a real-world application for face recognition. This typically involves capturing a face image, preprocessing it, and then feeding it into the neural network to obtain a feature vector. The feature vector is then compared to a database of known faces to determine the identity of the individual in the image.

Overall, human face recognition using DNNs is a complex process that requires a large amount of data, sophisticated neural network architectures, and careful preprocessing and training. However, with the increasing availability of large datasets and powerful computing resources, DNN-based face recognition systems have become increasingly accurate and effective in real-world applications.

Example- One example of human face recognition using DNNs is the FaceNet algorithm, which was developed by researchers at Google in 2015. FaceNet is a deep neural network that is trained to directly optimize the embedding of face images into a high-dimensional feature space, where distances between faces correspond to their similarity.

The FaceNet architecture consists of a deep convolutional neural network that takes a raw face image as input and produces a 128-dimensional feature vector as output. The network is trained on a large dataset of face images with labels that correspond to different individuals.

During training, the FaceNet network is optimized to minimize the distance between the feature vectors of images that depict the same person and maximize the distance between feature vectors of images that depict different people. This is done using a loss function called the triplet loss, which compares the distance between the feature vectors of an anchor image, a positive image (of the same person as the anchor), and a negative SNJB's Late Sau. K.B. Jain College Of Engineering image (of a different person). The goal is to minimize the distance between the anchor and positive images while maximizing the distance between the anchor and negative images.

After training, the FaceNet network can be used for face recognition by capturing a face image, preprocessing it, and then feeding it into the network to obtain a 128-dimensional feature vector. This feature vector is then compared to a database of known faces by computing the Euclidean distance between the feature vector and the feature vectors of the faces in the database. The closest matching face in the database is then returned as the recognized identity.

Overall, FaceNet is an example of a DNN-based face recognition system that has achieved high accuracy and robustness in real-world applications. It has been used in a variety of applications, including security system, access control, and social media tagging

Code :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2
# Define constants
img_height = 128
img_width = 128
batch_size = 32
epochs = 10
num_classes = 10
# Load the "CelebA" dataset
df = pd.read_csv('list_attr_celeba.csv')
SNJB's Late Sau. K.B. Jain College Of Engineering
df = df.sample(frac=1).reset_index(drop=True) # shuffle the dataset
df['image_id'] = df['image_id'].apply(lambda x: x[:-4]) # remove file extension from image IDs
df_train = df[:int(len(df)*0.8)] # 80% for training
df_val = df[int(len(df)*0.8):int(len(df)*0.9)] # 10% for validation
df_test = df[int(len(df)*0.9):] # 10% for testing

```

```

# Define data generators for training, validation, and testing sets
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_dataframe(
dataframe=df_train,
directory='img_align_celeba',
x_col='image_id',
y_col='Smiling',
target_size=(img_height, img_width),
batch_size=batch_size,
class_mode='binary')
val_generator = val_datagen.flow_from_dataframe(
dataframe=df_val,
directory='img_align_celeba',
x_col='image_id',
y_col='Smiling',
target_size=(img_height, img_width),
batch_size=batch_size,
SNJB's Late Sau. K.B. Jain College Of Engineering
class_mode='binary')
test_generator = test_datagen.flow_from_dataframe(
dataframe=df_test,
directory='img_align_celeba',
x_col='image_id',
y_col='Smiling',
target_size=(img_height, img_width),
batch_size=batch_size,

```

```

class_mode='binary')
# Define the neural network model
model = Sequential([
Conv2D(32, (3,3), activation='relu', input_shape=(img_height, img_width, 3)),
MaxPooling2D((2,2)),
Conv2D(64, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Conv2D(128, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Conv2D(128, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Flatten(),
Dropout(0.5),
Dense(512, activation='relu'),
Dense(num_classes, activation='sigmoid')
])
# Compile the model
model.compile(optimizer='adam',
SNJB's Late Sau. K.B. Jain College Of Engineering
loss='binary_crossentropy',
metrics=['accuracy'])
# Train the model
history = model.fit(train_generator,
epochs=epochs,
validation_data=val_generator)
# Evaluate the model on the test set

loss, accuracy = model.evaluate(test_generator)
print("Test accuracy:", accuracy)
# Predict the smiling attribute of a sample image
img = cv2.imread('sample_image.jpg')
img = cv2.resize(img, (img_height, img_width))
img = np.expand_dims(img, axis=0)

```

Conclusion: - In this way Human Face Recognition Implemented

Title: Implement Gender and Age Detection: predict if a person is a male or female and also their age

Project Title: Gender and Age Detection on UTKFace dataset

Objective: To build a deep neural network model that can recognize Gender and Age Detection on UTKFace dataset.

Hardware and Software Requirements:

Hardware:

- Multi-core CPU or a CPU with hyper-threading capability
- GPU (optional but recommended for faster training)
- Sufficient RAM to accommodate the dataset and model

Software:

- Python programming language
- Keras for deep learning
- Jupyter Notebook or any Python IDE for coding and experimentation

Theory:

Collect and prepare the dataset: In this case, we can use the "UTKFace" dataset which contains images of faces with their corresponding gender and age labels. We need to preprocess the data, like resizing the images to a uniform size, shuffling the dataset, and limit the age to a certain value (like 100 years).

Split the dataset: Split the dataset into training, validation, and testing sets. The usual split ratio is 80%, 10%, and 10%, respectively.

Define data generators: Define data generators for training, validation, and testing sets using the "ImageDataGenerator" class in Keras. This class provides data augmentation techniques that can improve the model's performance, such as rotation, zoom, and horizontal flip.

Define the neural network model: Define a convolutional neural network (CNN) model that takes the face images as input and outputs two values - the probability of being male and the predicted age. The model can have multiple convolutional and pooling layers followed by some dense layers.

Compile the model: Compile the model with appropriate loss and metrics for each output (gender and age). In this case, we can use binary cross-entropy loss for gender and mean squared error (MSE) for age.

Train the model: Train the model using the fit method of the model object. We need to pass the data generators for the training and validation sets, as well as the number of epochs and batch size.

Evaluate the model: Evaluate the model's performance on the testing set using the evaluate method of the model object. This will give us the accuracy and mean absolute error (MAE) of the model.

Predict the gender and age of a sample image: Load a sample image and preprocess it. We can use the

"cv2" library to read the image, resize it to the same size as the training images, and normalize it. Then, we can use the "predict" method of the model object to get the predicted gender and age.

Code :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2

# Define constants
img_height = 128
img_width = 128
batch_size = 32
epochs = 10

# Load the "UTKFace" dataset
df = pd.read_csv('UTKFace.csv')
df['age'] = df['age'].apply(lambda x: min(x, 100)) # limit age to 100
df = df.sample(frac=1).reset_index(drop=True) # shuffle the dataset
df['image_path'] = 'UTKFace/' + df['image_path']
df_train = df[:int(len(df)*0.8)] # 80% for training
df_val = df[int(len(df)*0.8):int(len(df)*0.9)] # 10% for validation
df_test = df[int(len(df)*0.9):] # 10% for testing

# Define data generators for training, validation, and testing sets
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_dataframe(
    dataframe=df_train,
    x_col='image_path',
    y_col=['male', 'age'],
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='raw')
val_generator = val_datagen.flow_from_dataframe(
    dataframe=df_val,
    x_col='image_path',
    y_col=['male', 'age'],
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='raw')
test_generator = test_datagen.flow_from_dataframe(
    dataframe=df_test,
    x_col='image_path',
    y_col=['male', 'age'],
    target_size=(img_height, img_width),
```

```

batch_size=batch_size,
class_mode='raw')
# Define the neural network model
model = Sequential([
Conv2D(32, (3,3), activation='relu', input_shape=(img_height, img_width, 3)),
MaxPooling2D((2,2)),
Conv2D(64, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Conv2D(128, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Conv2D(128, (3,3), activation='relu'),
MaxPooling2D((2,2)),
Flatten(),
Dropout(0.5),
Dense(512, activation='relu'),
Dense(2)
])
# Compile the model
model.compile(optimizer='adam',
loss={'dense_1': 'binary_crossentropy', 'dense_2': 'mse'},
metrics={'dense_1': 'accuracy', 'dense_2': 'mae'})
# Train the model
history = model.fit(train_generator,
epochs=epochs,
validation_data=val_generator)
# Evaluate the model on the test set
loss, accuracy, mae = model.evaluate(test_generator)
print("Test accuracy:", accuracy)
print("Test MAE:", mae)
# Predict the gender and age of a sample image
img = cv2.imread('sample_image.jpg')
Img

```

Conclusion: - In this way Gender Age Detection Implemented.